

Code Equivalence and Automorphism Problems for Codes

Jean-François Biasse¹, Alexandra V. Hostetler^{1*}, Anuvrat Jaindungarwal^{1*}

^{1*}Department of Mathematics and Statistics, University of South Florida,
4202 E Fowler Avenue, Tampa, 33620, FL, U.S.A..

*Corresponding author(s). E-mail(s): avelichehostetler@usf.edu; anuvrat@usf.edu;
Contributing authors: biasse@usf.edu;

Abstract

We study the complexity of the Code Equivalence problem and show that it is polynomially equivalent to several computational automorphism problems for codes. These problems ask for the cardinality (ACOUNT), an orbit partition (APART), and a generating set (AGEN) for the permutation automorphism group of a code. We present deterministic, polynomial-time reductions between Permutation Code Equivalence (PCE) and each of these problems, including a one-shot reduction from search-PCE to AGEN that makes a single oracle call. We present similar reductions between Linear Code Equivalence (LCE) and analogous problems for the monomial automorphism group of a code. All of our reductions work for any linear codes.

Keywords: code equivalence, automorphism group, permutation, monomial automorphism, linear isometry

1 Introduction

A (linear) code $\mathcal{C} \subseteq \mathbb{F}_q^n$ over a finite field $\mathbb{F}_q$ is a linear subspace of $\mathbb{F}_q^n$. The *Code Equivalence* (CE) problem asks whether two codes $\mathcal{C}_1$ and $\mathcal{C}_2$ are structurally equivalent in some way. Variants of this problem specify the type of equivalence: *Permutation Code Equivalence* (PCE) asks whether the codes are equivalent up to a permutation of coordinates of their codewords, while *Linear Code Equivalence* (LCE) asks whether the codes are equivalent up to a permutation and non-zero scaling of the codeword coordinates. Search variants of these problems (search-PCE and search-LCE) ask one to find such an equivalence map, if it exists.

A few prior works have studied the relationship between variants of CE. Biasse and Micheli in [1] proved a deterministic search-to-decision reduction for PCE that runs in time polynomial in n , the length of the input codes, and makes n^2 oracle calls. This result was extended by [2], and independently by [3], to a similar polynomial-time reduction from search-LCE to LCE. We restate their results below.

Lemma 1.1 (adapted from [1]). *There is a deterministic reduction from search-PCE on n -length codes to PCE which runs in $O(n^3 \log(q))$ time and makes at most n^2 oracle calls.*

Lemma 1.2 (adapted from [2]). *There is a deterministic reduction from search-LCE on n -length codes to LCE which runs in $O(n^3 \log(q))$ time and makes at most n^2 oracle calls.*

In [4], Cheraghchi, Shagrithaya, and Veliche (Hostetler) proved a Karp reduction from PCE to LCE that runs in time polynomial in n and polylogarithmic in q , the field size. The recent survey [5] details the current state-of-the-art of algorithms for solving variants of CE as well as the complexity.

PCE and LCE belong to a family of *isomorphism problems*, a general class of problems that ask whether two objects are structurally the same. These include the well-known Graph Isomorphism problem (GI) and Lattice Isomorphism problem (LIP). GI asks whether two graphs have an isomorphism that maps the vertices of one graph to the vertices of the other and preserves the edge structure. LIP asks whether two geometric lattices, described by two basis matrices, have an orthogonal linear transformation that maps one lattice to the other. In [6], Petrank and Roth showed a polynomial-time reduction from GI to PCE. Subsequent reductions from GI to CE variants were shown by Kaski and Östergård in [7] and Grochow in [8]. Bennett and Win in [9] later studied the relationship between GI, LIP, and variants of CE and showed a fine-grained dimension-preserving reduction from GI to LCE as well as a polynomial-time reduction from LCE over prime fields to LIP.

On its own, the GI problem has received much attention in the last half-century. Among others, Mathon [10] showed that the problem of recognizing whether two graphs are isomorphic is just as hard as counting all the isomorphisms between the two graphs. In particular, he proved that GI is polynomially equivalent to ACOUNT, which asks for the cardinality of a graph’s automorphism group, APART, which asks for a partition of the graph’s vertex set into orbits under automorphisms, and AGEN, which asks for a generating set of automorphisms that generate the graph’s automorphism group.

Besides isomorphism problems being interesting in their own right, they form the foundation of several modern cryptographic schemes. Variants of CE have been used as the basis of zero-knowledge protocols [11, 12] and the LESS identification and digital signature scheme [13–15]. LIP is used as the basis of the HAWK digital signature scheme [16, 17]. The use of CE in particular, as the hardness assumption of a post-quantum digital signature, was the motivation of many recent works on efficient algorithms for solving it [18–30]. Moreover, cryptosystems based on the hardness of isomorphism problems were presented to the NIST standardization process. The LESS scheme based on CE made it to the second round [31], while the HAWK proposal based on LIP made it to the third round [32]. Given the recent and increasing interest in basing cryptographic schemes on the hardness of code- and lattice-based problems, it is all the more crucial to thoroughly understand the complexity of these computational problems relative to other more well-studied problems. This will help inform future attempts to construct such cryptographic schemes.

Contributions. This work gives polynomial-time reductions between variants of CE and several automorphism problems for codes. We define ACOUNT, APART, and AGEN problems for codes analogously to those defined in [10] for graphs; these ask for the cardinality, an orbit partition, and a generating set of the permutation automorphism group of a given code, respectively. To match [10], we also define the ICOUNT problem for codes, which asks for the cardinality of the isomorphism group of permutations that maps one code to the other; this is closely related to ACOUNT. We give deterministic polynomial-time reductions that proves that PCE is equivalent to ACOUNT, APART, AGEN, and ICOUNT.

In this work, we also prove a *one-shot* deterministic polynomial-time reduction from search-PCE to AGEN, where the algorithm solves search-PCE by making a single oracle call to AGEN. To our knowledge, there is no graph analog of this reduction.

We define analogous problems for the *monomial automorphism* group of a code, which allows non-zero scaling of codeword coordinates in addition to permutation; these problems are MACOUNT, MAPART, MAGEN, and MICOUNT. We prove analogous deterministic polynomial-time reductions for all of the reductions for PCE, showing that LCE is equivalent to these monomial automorphism problems.

A few years ago, Grochow mentioned in [33] the possibility of reducing PCE to AGEN for codes that cannot be decomposed into a direct sum of other codes. This, together with the analogous result for LCE, was stated informally in [5]. Our work provides formal proofs of these results with pseudocode for the algorithms and tight analysis of the runtime and number of oracle calls. Moreover, our reductions work for *any* linear codes.

Technical Overview. The isomorphism and automorphism problems for codes are defined formally in Section 2. We present auxiliary lemmas that describe the relationship between code isomorphisms and automorphisms, and related properties.

Our reductions rely on the inherent structure of the given code(s), namely its decomposability into a direct sum of other codes and the properties of the generator matrix used to describe the code. In Section 3, we present a framework for code decomposition and show that any code can be uniquely decomposed into a direct sum of indecomposable codes up to a permutation of the summands and

Note that the support graph of a code is unique. This follows immediately from the fact that a linear code has a uniquely determined generator matrix in reduced row echelon form (RREF). Hence, we use $\Gamma(\mathcal{C})$ to denote the unique support graph of a code $\mathcal{C}$.

For any $[n, k]_q$ -code $\mathcal{C}$, constructing its support graph from its $k \times n$ generator matrix takes $O(kn^2 \log(q))$ time. This is because adding all the edges requires iterating through all k rows, each of which can contain up to n non-zero entries in $\mathbb{F}_q$.

Definition 2.1 (Direct sum of codes). *The direct sum of two codes $\mathcal{C}_1, \mathcal{C}_2$ generated by matrices $\mathbf{G}_1 \in \mathbb{F}_q^{k_1 \times n_1}$ and $\mathbf{G}_2 \in \mathbb{F}_q^{k_2 \times n_2}$, respectively, is the code $\mathcal{C} := \mathcal{C}_1 \oplus \mathcal{C}_2 \subseteq \mathbb{F}_q^n$ generated by the matrix*

$$\mathbf{G} := \begin{bmatrix} \mathbf{G}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{G}_2 \end{bmatrix} \in \mathbb{F}_q^{k \times n},$$

where $n := n_1 + n_2$ and $k := k_1 + k_2$. $\mathcal{C}$ contains all vectors of the form $[\mathbf{c}_1 \ \mathbf{c}_2]$, where $\mathbf{c}_1 \in \mathcal{C}_1$ and $\mathbf{c}_2 \in \mathcal{C}_2$.

Throughout this paper, we only take the direct sum of two codes in the context of solving PCE or LCE. For this use, we can assume without loss of generality that the generator matrices $\mathbf{G}_1$ and $\mathbf{G}_2$ of the input codes do not contain all-zero columns; this follows from Lemma 3.4 in [4]. For any such direct sum code $\mathcal{C} = \mathcal{C}_1 \oplus \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, we define X_1 and X_2 to be the disjoint supports of the subcodes $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively, i.e. $X_i := \cup_{\mathbf{c} \in \mathcal{C}_i} \text{supp}(\mathbf{c}) \subseteq [n]$. We call X_1 and X_2 the *index sets of $\mathcal{C}_1$ and $\mathcal{C}_2$ embedded in X* . By definition of direct sum, these have cardinality $|X_i| = n_i$. Denoting the projection of $\mathcal{C}$ onto the set X_i by $\mathcal{C}|_{X_i}$, we can write $\mathcal{C} = \mathcal{C}|_{X_1} \oplus \mathcal{C}|_{X_2}$. We sometimes abuse notation and use $\mathcal{C}_i$ to denote the embedded code $\{\mathbf{0}\} + \dots + \mathcal{C}_i + \dots + \{\mathbf{0}\}$ in a direct sum $\mathcal{C}_1 \oplus \dots \oplus \mathcal{C}_i \oplus \dots \oplus \mathcal{C}_m$.

Definition 2.2 (Bridging Element). *Let $\mathcal{C}$ be a code whose index set X has a non-trivial partition $X = Y \cup Z$. A codeword $\mathbf{c} \in \mathcal{C}$ is a bridging element with respect to (Y, Z) if $\text{supp}(\mathbf{c}) \cap Y \neq \emptyset \neq \text{supp}(\mathbf{c}) \cap Z$ and there is no pair $\mathbf{u}, \mathbf{v} \in \mathcal{C}$ such that $\mathbf{c} = \mathbf{u} + \mathbf{v}$ and $\text{supp}(\mathbf{u}) \subseteq Y$ and $\text{supp}(\mathbf{v}) \subseteq Z$.*

For example, consider the binary repetition code $\mathcal{C} = \{00, 11\}$ and the partition $Y = \{1\}$ and $Z = \{2\}$, the codeword 11 is a bridging element since the code does not have the elements 10 and 01 required to decompose it across the partition.

2.1 Computational Code Problems

We now define the computational problems for codes that are the focus of study in this paper.

Two codes $\mathcal{C}_1$ and $\mathcal{C}_2$ are called *permutation equivalent* if the coordinates of $\mathcal{C}_1$ can be permuted to obtain $\mathcal{C}_2$. They are said to be *isometric* if there exists an $\mathbb{F}_q$ -linear isomorphism between them that strictly preserves the Hamming weight of each codeword. We formalize and generalize these notions in the following two problems.

Definition 2.3 (PCE). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Permutation Code Equivalence (PCE) problem is the following decision problem: Given a pair of generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1 = \mathcal{C}(\mathbf{G}_1), \mathcal{C}_2 = \mathcal{C}(\mathbf{G}_2) \subseteq \mathbb{F}_q^n$, decide whether there exists a permutation isomorphism $\pi \in \mathcal{S}_n$ such that $\pi(\mathcal{C}_1) = \mathcal{C}_2$.*

Definition 2.4 (LCE). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Linear Code Equivalence (LCE) problem is the following decision problem: Given a pair of generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1 = \mathcal{C}(\mathbf{G}_1), \mathcal{C}_2 = \mathcal{C}(\mathbf{G}_2) \subseteq \mathbb{F}_q^n$, decide whether there exists a linear isometry (a.k.a. monomial isomorphism) $\mu = (\mathbf{v}, \pi) \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$ such that $\mu(\mathcal{C}_1) = \mathcal{C}_2$.*

When two codes $\mathcal{C}_1, \mathcal{C}_2$ are permutation equivalent, their generator matrices $\mathbf{G}_1, \mathbf{G}_2$ form a PCE instance; we denote this by $\mathcal{C}_1 \cong \mathcal{C}_2$ and say simply that they are *equivalent*. If they are *monomially equivalent*, they form an LCE instance, and we denote this by $\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$. We will use the following observation about the column frequency of generator matrices of equivalent codes.

Lemma 2.1 (Lemma 3.3 , [4]). *Let k and n be positive integers and $\mathbb{F}_q$ be a finite field. If any two matrices $\mathbf{G}, \mathbf{H} \in \mathbb{F}_q^{k \times n}$ form a PCE instance, then no column of $\mathbf{G}$ appears in $\mathbf{G}$ more times than a column of $\mathbf{H}$ appears in $\mathbf{H}$.*

Lemma 2.2 (Remark 1 , search to decision LCE). *Let k and n be positive integers and $\mathbb{F}_q$ be a finite field. If any two matrices $\mathbf{G}, \mathbf{H} \in \mathbb{F}_q^{k \times n}$ form an LCE instance, then the columns of $\mathbf{G}$ have the same frequency as the columns of $\mathbf{H}$, up to non-zero scaling.*

The following lemma states that isometry and monomial equivalence are equivalent notions. We thus use both terms interchangeably throughout the paper.

Lemma 2.3 ([34], Corollary 1). *Any two codes $\mathcal{C}_1$ and $\mathcal{C}_2$ are monomially equivalent ($\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$) if and only if $\mathcal{C}_1$ is isometric to $\mathcal{C}_2$.*

Recall that *isometric* here refers to the existence of an $\mathbb{F}_q$ -linear isometry.

We will also discuss the search variants of PCE and LCE, which ask to find the isomorphism or linear isometry between a given pair of codes, if it exists.

Definition 2.5 (Search-PCE). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, search-PCE is the following search problem: Given a pair of generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1 = \mathcal{C}(\mathbf{G}_1), \mathcal{C}_2 = \mathcal{C}(\mathbf{G}_2) \subseteq \mathbb{F}_q^n$, find a permutation $\pi \in \mathcal{S}_n$ for which $\pi(\mathcal{C}_1) = \mathcal{C}_2$, if it exists.*

Definition 2.6 (Search-LCE). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, search-LCE is the following search problem: Given a pair of generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1 = \mathcal{C}(\mathbf{G}_1), \mathcal{C}_2 = \mathcal{C}(\mathbf{G}_2) \subseteq \mathbb{F}_q^n$, find a monomial permutation $\mu \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$ for which $\mu(\mathcal{C}_1) = \mathcal{C}_2$, if it exists.*

Note that permutation equivalence implies monomial equivalence. It is also natural to ask for information about the set of isomorphisms or linear isometries between a given pair of codes. For any pair of codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, we denote the set of all permutation isomorphisms between them by

$$\text{Iso}(\mathcal{C}_1, \mathcal{C}_2) = \{\pi \in \mathcal{S}_n \mid \pi(\mathcal{C}_1) = \mathcal{C}_2\}. \quad (2.1)$$

Similarly, we denote the set of all linear isometries between them by

$$\text{Isom}(\mathcal{C}_1, \mathcal{C}_2) = \{\mu \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n \mid \mu(\mathcal{C}_1) = \mathcal{C}_2\}. \quad (2.2)$$

It is immediate that $\text{Iso}(\mathcal{C}_1, \mathcal{C}_2) \subseteq \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$. We are interested in computing the cardinalities of these sets, as defined in the following problems.

Definition 2.7 (ICOUNT, MICOUNT). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Isomorphism Counting (ICOUNT) (resp. Monomial Isomorphism Counting (MICOUNT)) problem is the following search problem: Given two generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, compute the cardinality $|\text{Iso}(\mathcal{C}_1, \mathcal{C}_2)|$ (resp. $|\text{Isom}(\mathcal{C}_1, \mathcal{C}_2)|$) of the set of (resp. monomial) isomorphisms between $\mathcal{C}_1$ and $\mathcal{C}_2$. If $\mathcal{C}_1 \not\cong \mathcal{C}_2$ (resp. $\mathcal{C}_1 \not\cong_{\mathcal{M}} \mathcal{C}_2$), output 0.*

Any permutation on the coordinates of a code $\mathcal{C} \subseteq \mathbb{F}_q^n$ induces a map on its codewords; for this reason, we use $\pi(\mathcal{C})$ to denote the code obtained by applying the permutation $\pi \in \mathcal{S}_n$ to $\mathcal{C}$. We denote the group of permutation automorphisms of $\mathcal{C}$ by

$$\text{Aut}(\mathcal{C}) := \{\pi \in \mathcal{S}_n \mid \pi(\mathcal{C}) = \mathcal{C}\} \subseteq \mathcal{S}_n. \quad (2.3)$$

This group corresponds to the group of permutation matrices $\mathbf{P}$ that satisfy $\mathbf{SGP} = \mathbf{G}$ for any generator $\mathbf{G}$ of the code and some invertible change-of-basis matrix $\mathbf{S}$. Similarly, we denote the group of monomial automorphisms of $\mathcal{C}$ by

$$\text{MAut}(\mathcal{C}) := \{\mu \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n \mid \mu(\mathcal{C}) = \mathcal{C}\} \subseteq (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n. \quad (2.4)$$

Note that $\text{Aut}(\mathcal{C}) \subseteq \text{MAut}(\mathcal{C})$. Any monomial transformation $(\mathbf{v}_\mu, \mu) \in \text{MAut}(\mathcal{C})$ acts on the ambient space $\mathbb{F}_q^n$. So, for any subcode $\mathcal{A} \subseteq \mathcal{C}$, the set $\mu(\mathcal{A})$ is the image of the subspace $\mathcal{A}$ under this transformation, i.e. $\mu(\mathcal{A}) := \{\mu(\mathbf{a}) \mid \mathbf{a} \in \mathcal{A}\} \subseteq \mu(\mathcal{C}) \subseteq \mathbb{F}_q^n$. The group of monomial automorphisms $\text{MAut}(\mathcal{C})$ corresponds to the group of monomial matrices $\mathbf{M}$ that satisfy $\mathbf{S}\mathbf{G}\mathbf{M} = \mathbf{G}$ for any generator $\mathbf{G}$ of $\mathcal{C}$ and for some invertible change-of-basis matrix $\mathbf{S}$.

Isomorphisms between two codes are closely related to automorphisms of their direct sum code. In the following lemma, we show that the set of all isomorphisms between two codes is a coset of the automorphism group of each code.

Lemma 2.4 (folklore). *For any pair of equivalent codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, let $\text{Iso}(\mathcal{C}_1, \mathcal{C}_2)$ be the set of all isomorphisms from $\mathcal{C}_1$ to $\mathcal{C}_2$. For any fixed isomorphism $\pi \in \text{Iso}(\mathcal{C}_1, \mathcal{C}_2)$,*

$$\text{Iso}(\mathcal{C}_1, \mathcal{C}_2) = \text{Aut}(\mathcal{C}_2) \circ \pi = \{\alpha \circ \pi \mid \alpha \in \text{Aut}(\mathcal{C}_2)\}.$$

Furthermore, $|\text{Iso}(\mathcal{C}_1, \mathcal{C}_2)| = |\text{Aut}(\mathcal{C}_2)|$.

Lemma 2.5. *For any pair of monomially equivalent codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, let $\text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$ be the set of all linear isometries from $\mathcal{C}_1$ to $\mathcal{C}_2$. For any fixed linear isometry $\mu \in \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$,*

$$\text{Isom}(\mathcal{C}_1, \mathcal{C}_2) = \text{MAut}(\mathcal{C}_2) \circ \mu = \{\alpha \circ \mu \mid \alpha \in \text{MAut}(\mathcal{C}_2)\}.$$

Furthermore, $|\text{Isom}(\mathcal{C}_1, \mathcal{C}_2)| = |\text{MAut}(\mathcal{C}_2)|$.

Proof Since $\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$, there exists at least one isomorphism $\pi \in \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$. First, we show that $\text{MAut}(\mathcal{C}_2) \circ \pi \subseteq \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$. Consider any monomial automorphism $\alpha \in \text{MAut}(\mathcal{C}_2)$. Since $\pi(\mathcal{C}_1) = \mathcal{C}_2$ and $\alpha(\mathcal{C}_2) = \mathcal{C}_2$, the composition of π and α satisfies $\alpha \circ \pi(\mathcal{C}_1) = \mathcal{C}_2$. Hence, $\alpha \circ \pi \in \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$. Now, we show that $\text{Isom}(\mathcal{C}_1, \mathcal{C}_2) \subseteq \text{MAut}(\mathcal{C}_2) \circ \pi$. Consider any isomorphism $\psi \in \text{Isom}(\mathcal{C}_1, \mathcal{C}_2)$. Since π^{-1} is a monomial isomorphism from $\mathcal{C}_2$ to $\mathcal{C}_1$ and $\psi(\mathcal{C}_1) = \mathcal{C}_2$, their composition satisfies $\psi \circ \pi^{-1}(\mathcal{C}_2) = \mathcal{C}_2$. Hence, $\psi \circ \pi^{-1} \in \text{MAut}(\mathcal{C}_2)$ and $\psi = \psi \circ \pi^{-1} \circ \pi \in \text{MAut}(\mathcal{C}_2) \circ \pi$. Therefore, $\text{Isom}(\mathcal{C}_1, \mathcal{C}_2) = \text{MAut}(\mathcal{C}_2) \circ \pi$. Moreover, by the properties of group actions, we obtain that $|\text{Isom}(\mathcal{C}_1, \mathcal{C}_2)| = |\text{MAut}(\mathcal{C}_2)|$. $\square$

The following automorphism problems were presented in [10] for graphs; we define their code counterparts here. These problems ask for various descriptors of the automorphism group of a given code, such as its cardinality, a set of generators, and an orbit partition.

For any coordinate $i \in [n]$ of code $\mathcal{C} \subseteq \mathbb{F}_q^n$, we denote its *orbit* under the automorphism group $\text{Aut}(\mathcal{C})$ by $O_i := \{j \in [n] \mid \alpha(i) = j \text{ for some } \alpha \in \text{Aut}(\mathcal{C})\}$. Note that trivially $i \in O_i$. We use a similar notation for *monomial orbits* of coordinates under the monomial automorphism group $\text{MAut}(\mathcal{C})$.

Definition 2.8 (APART, MAPART). *For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Automorphism Partition (APART) (resp. Monomial Automorphism Partition (MAPART)) problem is the following search problem: Given a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a code $\mathcal{C}$, find the partition of its index set $[n]$ into (resp. monomial) orbits $O_1, \dots, O_n \subseteq [n]$, where $O_i = O_j$ if and only if there exists $\alpha \in \text{Aut}(\mathcal{C})$ (resp. $\alpha = (\mathbf{v}_\alpha, \pi_\alpha) \in \text{MAut}(\mathcal{C})$) such that $\alpha(i) = j$ (resp. $\pi_\alpha(i) = j$).*

Remark 2.1. *The monomial orbit of an index is defined only with respect to the permutation component of any monomial automorphism in $\text{MAut}(\mathcal{C})$. Defining orbits with respect to index-scalar pairs is superfluous, because for any coordinate permutation, scaling every coordinate by the same scalar gives a monomial automorphism. To see why, consider any generator matrix $\mathbf{G}$. Multiplication by any non-zero scalar $\lambda \in \mathbb{F}_q^*$ gives the identity $\mathbf{D}\mathbf{G} = \mathbf{G}(\lambda\mathbf{I}_n)$. Isolating $\mathbf{G}$ on the right yields $(\lambda^{-1}\mathbf{I}_k)\mathbf{G}(\lambda\mathbf{I}_n) = \mathbf{G}$. The diagonal matrices $\mathbf{S} = \lambda^{-1}\mathbf{I}_k$ and $\mathbf{M} = \lambda\mathbf{I}_n$ satisfy the monomial equivalence condition $\mathbf{S}\mathbf{G}\mathbf{M} = \mathbf{G}$. We can decompose $\mathbf{M}$ into the permutation matrix $\mathbf{P} = \mathbf{I}_n$ and diagonal matrix $\mathbf{D} = \text{diag}(\lambda, \dots, \lambda)$. The associated automorphism maps every coordinate i to itself, since $\mathbf{P}_{i,i} = 1$, and multiplies each i -th column of $\mathbf{G}$ by λ , since $\mathbf{D}_{i,i} = \lambda$. Therefore, any index-scalar pair (i, λ) would have the same orbit as $(i, 1)$ for all $\lambda \in \mathbb{F}_q^*$.*

Definition 2.9 (ACOUNT, MACOUNT). For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Automorphism Counting (ACOUNT) (resp. Monomial Automorphism Counting (MACOUNT)) problem is the following computational problem: Given a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a code $\mathcal{C} = \mathcal{C}(\mathbf{G})$, compute the order of its permutation automorphism group $|\text{Aut}(\mathcal{C})|$ (resp. monomial automorphism group $|\text{MAut}(\mathcal{C})|$).

Definition 2.10 (AGEN, MAGEN). For $n, k \in \mathbb{N}$ and finite field $\mathbb{F}_q$, the Automorphism Generation (AGEN) (resp. Monomial Automorphism Generation (MAGEN)) problem is the following search problem: Given a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a code $\mathcal{C}$, find a set of permutations (resp. linear isometries) S that generates the group $\langle S \rangle = \text{Aut}(\mathcal{C})$ (resp. $\langle S \rangle = \text{MAut}(\mathcal{C})$).

It is natural to define similar computational problems relating to isomorphisms of a given pair of codes. The ICOUNT problem was defined in [10] as the analog of ACOUNT. The isomorphism analogs of AGEN and APART arise as a direct consequence of Lemma 2.4.

Suppose we want to find the set of all isomorphisms between two equivalent codes $\mathcal{C}_1$ and $\mathcal{C}_2$. If we can find a generating set for the automorphism group $\text{Aut}(\mathcal{C}_2)$, it is enough to find one isomorphism π from $\mathcal{C}_1$ to $\mathcal{C}_2$ to obtain the set of isomorphisms $\text{Iso}(\mathcal{C}_1, \mathcal{C}_2)$. In this way, this problem of finding all isomorphisms between two equivalent codes trivially reduces to solving AGEN and search-PCE.

Similarly, if we want to find the orbits of the indices of a code $\mathcal{C}_1$ under any isomorphism from $\mathcal{C}_1$ to $\mathcal{C}_2$, it is enough to find the orbit partition of the index set of $\mathcal{C}_1$ and an isomorphism π from $\mathcal{C}_1$ to $\mathcal{C}_2$. In this way, this problem of finding an orbit partition of the index set of two equivalent codes under isomorphisms trivially reduces to solving APART and search-PCE.

2.2 Stabilizer Chains and Generating Sets

Sims introduced in [35] the notion of a stabilizer chain determined by a base as a data structure for efficiently manipulating permutation groups. We define similar structures in Section 3.4 and use them for our reductions in Sections 5.3 and 6.3. For reference, we include the definitions and some key properties of the original notion here.

Definition 2.11 (Base, Pointwise Stabilizer chain). For any positive integer n and permutation group $G \leq \mathcal{S}_n$, a sequence of elements $B = \{\beta_1, \dots, \beta_s\} \subseteq [n]$ is called a base if the only element of G that fixes B is the identity map. The (pointwise) stabilizer chain for G determined by base B is the chain of nested subgroups

$$G =: G^{(0)} \geq G^{(1)} \geq \dots \geq G^{(s)} = \langle \text{id} \rangle,$$

where $G^{(i)} := G_{\beta_1, \dots, \beta_i}$ is the subgroup that stabilizes $\{\beta_1, \dots, \beta_i\}$ pointwise, for each $i \in [s]$.

Lemma 2.6 (adapted from [36]). For any $n \in \mathbb{N}$ and permutation group $G \leq \mathcal{S}_n$ with stabilizer chain $G =: G^{(0)} \geq G^{(1)} \geq \dots \geq G^{(s)} = \langle \text{id} \rangle$,

$$|G| = \prod_{i=0}^{s-1} |G^{(i)} : G^{(i+1)}|.$$

Proof By Lagrange's Theorem, the order of each subgroup $G^{(i)}$ can be written as $|G^{(i)}| = |G^{(i)} : G^{(i+1)}| \cdot |G^{(i+1)}|$. Rewriting each term in the product and canceling like terms gives the claimed equality. $\square$

Definition 2.12 (Strong Generating Set). For any $n \in \mathbb{N}$ and group $G \leq \mathcal{S}_n$, let B be a base that defines the stabilizer chain $G =: G^{(0)} \geq G^{(1)} \geq \dots \geq G^{(s)} = \langle \text{id} \rangle$. A strong generating set for G with respect to base B is a set S that generates $\langle S \rangle = G$ and satisfies the property $\langle S \cap G^{(i)} \rangle = G^{(i)}$ for all $i \in [s]$.

Lemma 2.7. For any groups $H \leq G$, let S_H be a generating set for H and $R_{G/H}$ be the set of coset representatives of the quotient G/H . Then, G can be expressed as

$$G = \langle S_H \cup R_{G/H} \rangle.$$

Before showing how we adapt the structure of a stabilizer set for our purpose of counting automorphisms of a code, we need to introduce code decomposition.

3 Code Decomposition

For the reductions between GI and graph automorphism problems, Mathon in [10] relies on the assumption that the input graphs are connected. Even if a graph unconnected graph, the same reductions can be performed on its complement, which is a connected graph. The code analog of connectedness is its decomposability into a direct sum of indecomposable codes. In this section, we define code (in)decomposability and prove some key properties that are essential for our reductions. To ensure that our reductions work for *any* codes and not just indecomposable codes, we provide an efficient method of decomposing a linear code.

Definition 3.1 (Decomposable Codes). *A code $\mathcal{C}$ is (direct sum) decomposable if it is permutation (resp. monomially) equivalent to a direct sum of two or more codes, i.e. $\mathcal{C} \cong \mathcal{C}_1 \oplus \dots \oplus \mathcal{C}_m$ (resp. $\mathcal{C} \cong_{\mathcal{M}} \mathcal{C}_1 \oplus \dots \oplus \mathcal{C}_m$) for some codes $\mathcal{C}_1, \dots, \mathcal{C}_m$. Otherwise, we say that $\mathcal{C}$ is indecomposable.*

3.1 Existence and Uniqueness

The existence and uniqueness of the direct sum decomposition of linear codes was originally established by Slepian in [37]. We restate these facts here and extend them to monomial equivalence.

Lemma 3.1 ([37], Theorem 2). *Any $[n, k]$ -code $\mathcal{C}$ is equivalent to a direct sum of indecomposable codes, i.e.*

$$\mathcal{C} \cong \mathcal{A}_1 \oplus \dots \oplus \mathcal{A}_m,$$

for some indecomposable $\mathcal{A}_1, \dots, \mathcal{A}_m$. Furthermore, this decomposition is unique up to permutation equivalence and reordering of the summands, i.e. if $\mathcal{C} \cong \mathcal{B} \oplus \dots \oplus \mathcal{B}_{m'}$, for indecomposable codes $\mathcal{B}_1, \dots, \mathcal{B}_{m'}$, then $m = m'$ and $\mathcal{B}_i \cong \mathcal{A}_{\sigma(i)}$ for all $i \in [m]$ for some index permutation $\sigma : [m] \rightarrow [m]$.

Indecomposable codes have the following useful property.

Lemma 3.2. *A code $\mathcal{C}$ is indecomposable if and only if for every non-trivial partition of its index set $X = Y \sqcup Z$, there exists at least one bridging element $\mathbf{c} \in \mathcal{C}$ with respect to (Y, Z) .*

Proof ($\Rightarrow$) Let $\mathcal{C}$ be indecomposable. Assume for contradiction that there exists a non-trivial partition $X = Y \sqcup Z$ for which there is no bridging element in $\mathcal{C}$. Then, every codeword $\mathbf{c} \in \mathcal{C}$ whose support intersects both Y and Z can be expressed as a sum of two vectors with supports in Y and Z , i.e. $\mathbf{c} = \mathbf{u} + \mathbf{v}$ for some $\mathbf{u}, \mathbf{v} \in \mathcal{C}$ such that $\text{supp}(\mathbf{u}) \subseteq Y$ and $\text{supp}(\mathbf{v}) \subseteq Z$. Hence, $\mathcal{C}$ can be written as a direct sum $\mathcal{C} = \mathcal{C}|_Y \oplus \mathcal{C}|_Z$, which contradicts the indecomposability of $\mathcal{C}$.

($\Leftarrow$) We prove the contrapositive. Suppose $\mathcal{C}$ is decomposable and let X denote its index set. Then $\mathcal{C} \cong \mathcal{C}_1 \oplus \mathcal{C}_2$ for some codes $\mathcal{C}_1, \mathcal{C}_2$. By definition of direct sum, $\mathcal{C}_1, \mathcal{C}_2$ are supported index sets Y, Z that form a strict, non-trivial partition $X = Y \sqcup Z$. Also every codeword $\mathbf{c} \in \mathcal{C}$ can be uniquely written as $\mathbf{c} = [\mathbf{u} \ \mathbf{v}]$ for some $\mathbf{u} \in \mathcal{C}_1$ and $\mathbf{v} \in \mathcal{C}_2$. As a result of the partition, we have $\text{supp}(\mathbf{u}) \subseteq Y$ and $\text{supp}(\mathbf{v}) \subseteq Z$. Consequently, for this partition (Y, Z) , every codeword in $\mathcal{C}$ decomposes into a sum of elements, each with non-intersecting support across the partition. Thus, there is no bridging element with respect to (Y, Z) . This proves the contrapositive claim. $\square$

We prove an analogous statement to Lemma 3.1 for monomial equivalence. To do this, we use the notion of the *closure of a code* introduced by Sendrier and Simos in [38]. Fix an ordering of the elements in $\mathbb{F}_q^*$. For any code $\mathcal{C} \subseteq \mathbb{F}_q^n$, its closure $\tilde{\mathcal{C}} \subseteq \mathbb{F}_q^{n(q-1)}$ is the set defined by taking every codeword and multiplying each of its n coordinates with all the non-zero field elements to produce a new vector of length $n(q-1)$. Formally,

$$\tilde{\mathcal{C}} := \{(\alpha_1 \cdot c_1, \dots, \alpha_{q-1} \cdot c_1)_{i \in [n]} \mid \mathbf{c} = (c_1, \dots, c_n) \in \mathcal{C}, \alpha_1, \dots, \alpha_{q-1} \in \mathbb{F}_q^*\} \subseteq \mathbb{F}_q^{n(q-1)}.$$

Lemma 3.3 (Theorem 1 [38]). *For any two codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, $\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$ if and only if $\tilde{\mathcal{C}}_1 \cong \tilde{\mathcal{C}}_2$.*

Lemma 3.4. *The closure operation has the following properties.*

- (i) *It distributes over direct sums, i.e. any direct sum code $\mathcal{C} = \bigoplus_{i=1}^m \mathcal{A}_i$ has closure $\tilde{\mathcal{C}} = \bigoplus_{i=1}^m \tilde{\mathcal{A}}_i$.*

(ii) $\mathcal{C}$ is indecomposable if and only if its closure $\widetilde{\mathcal{C}}$ is indecomposable.

Proof (i) Let $\mathcal{C} = \bigoplus_{i=1}^m \mathcal{A}_i$ be a direct sum of indecomposable codes $\mathcal{A}_1, \dots, \mathcal{A}_m$ and let $\mathbf{G}$ be a $k \times n$ generator matrix for $\mathcal{C}$. Then, by definition of direct sum code, $\mathbf{G}$ can be written in block-diagonal form, where each i -th block $\mathbf{G}_i$ on the diagonal is a generator matrix for $\mathcal{A}_i$, for all $i \in [m]$. By definition of closure, a generator matrix for $\widetilde{\mathcal{C}}$ can be obtained by replacing each column $\mathbf{g}_j$ of $\mathbf{G}$ with the matrix containing its non-zero scalar multiples, $[\alpha_1 \mathbf{g}_j \ \dots \ \alpha_{q-1} \mathbf{g}_j]$, where $\mathbb{F}_q^* = \{\alpha_1, \dots, \alpha_{q-1}\}$. Since scalar multiplication leaves any 0 entry unchanged, this column expansion produces a $k \times n(q-1)$ matrix $\widetilde{\mathbf{G}}$ in block-diagonal form, where the i -th block on the diagonal is exactly the expansion of $\mathbf{G}_i$, for all $i \in [m]$. Therefore, by definition of direct sum, $\widetilde{\mathbf{G}}$ generates the direct sum code $\bigoplus_{i=1}^m \widetilde{\mathcal{A}}_i$.

(ii) Let $X = [n]$ be the index set of the indecomposable code $\mathcal{C}$. By definition, the closure operation expands each coordinate of the code into a subspace of its scalar multiples. Hence, every coordinate of $\mathcal{C}$ with index $j \in X$ corresponds to a projective class in $\widetilde{\mathcal{C}}$ whose index set is $E_j = \{j(q-1) - q + 2, \dots, j(q-1)\}$.

Suppose for contradiction that $\mathcal{C}$ is indecomposable and its closure $\widetilde{\mathcal{C}}$ is decomposable. Then by Lemma 3.2, the index set of $\widetilde{\mathcal{C}}$ has a non-trivial partition $Y' \sqcup Z' = [n(q-1)]$ such that $\widetilde{\mathcal{C}} = \widetilde{\mathcal{C}}|_{Y'} \oplus \widetilde{\mathcal{C}}|_{Z'}$. For any $j \in X$, all coordinates of $\widetilde{\mathcal{C}}$ with index in E_j are scalar multiples of each other, and so are linearly dependent. Then, by definition of direct sum, these coordinates cannot be expressed in terms of more than one indecomposable summand of $\widetilde{\mathcal{C}}$. Hence, the entire set E_j must be contained either entirely within Y' or entirely within Z' . This implies that there must be a non-trivial partition $Y \sqcup Z = [n]$ of the original index set, such that $j \in Y \iff E_j \subseteq Y'$ and $j \in Z \iff E_j \subseteq Z'$. Then for this partition, we have $\mathcal{C} = \mathcal{C}|_Y \oplus \mathcal{C}|_Z$, which contradicts the assumption that $\mathcal{C}$ is indecomposable. The converse follows by a symmetric argument. $\square$

Lemma 3.5. Every code $\mathcal{C} \subseteq \mathbb{F}_q^n$ is monomially equivalent to a direct sum of indecomposable codes, i.e.

$$\mathcal{C} \cong_{\mathcal{M}} \mathcal{A}_1 \oplus \dots \oplus \mathcal{A}_m,$$

for some indecomposable $\mathcal{A}_1, \dots, \mathcal{A}_m$. Furthermore, this decomposition is unique up to monomial equivalence and reordering of the summands, i.e. if $\mathcal{C} \cong_{\mathcal{M}} \mathcal{B} \oplus \dots \oplus \mathcal{B}_{m'}$, for indecomposable codes $\mathcal{B}_1, \dots, \mathcal{B}_{m'}$, then $m = m'$ and $\mathcal{B}_i \cong_{\mathcal{M}} \mathcal{A}_{\sigma(i)}$ for all $i \in [m]$ for some index permutation $\sigma : [m] \rightarrow [m]$.

Proof (Existence) By Lemma 3.1, every code $\mathcal{C}$ is equivalent to a direct sum of indecomposable codes, i.e. $\mathcal{C} \cong \mathcal{A}_1 \oplus \dots \oplus \mathcal{A}_m$ for some indecomposable codes $\mathcal{A}_1, \dots, \mathcal{A}_m$. Because permutation equivalence implies monomial equivalence, it trivially follows that $\mathcal{C} \cong_{\mathcal{M}} \mathcal{A}_1 \oplus \dots \oplus \mathcal{A}_m$.

(Uniqueness) Let $\mathcal{C}_A := \mathcal{A}_1 \oplus \dots \oplus \mathcal{A}_m$ and $\mathcal{C}_B := \mathcal{B}_1 \oplus \dots \oplus \mathcal{B}_{m'}$. Suppose $\mathcal{C} \cong_{\mathcal{M}} \mathcal{C}_A$ and $\mathcal{C} \cong_{\mathcal{M}} \mathcal{C}_B$. By transitivity, $\mathcal{C}_A \cong_{\mathcal{M}} \mathcal{C}_B$. By Lemma 3.3, $\mathcal{C}_A \cong_{\mathcal{M}} \mathcal{C}_B$ implies that their closure codes satisfy $\widetilde{\mathcal{C}}_A \cong_{\mathcal{M}} \widetilde{\mathcal{C}}_B$. By Lemma 3.4, we have $\mathcal{C}_A = \widetilde{\mathcal{A}}_1 \oplus \dots \oplus \widetilde{\mathcal{A}}_m$ and $\mathcal{C}_B = \widetilde{\mathcal{B}}_1 \oplus \dots \oplus \widetilde{\mathcal{B}}_{m'}$, where each summand $\widetilde{\mathcal{A}}_i$ and $\widetilde{\mathcal{B}}_j$ is an indecomposable code. Substituting these direct sum identities into the equivalence yields

$$\widetilde{\mathcal{C}} = \widetilde{\mathcal{A}}_1 \oplus \dots \oplus \widetilde{\mathcal{A}}_m \cong \widetilde{\mathcal{B}}_1 \oplus \dots \oplus \widetilde{\mathcal{B}}_{m'} = \widetilde{\mathcal{C}}_B.$$

By Lemma 3.1, the decomposition of a code into indecomposable summands is unique up to permutation equivalence. This implies that the number of summands is identical, so $m = m'$, and there exists a bijection $\sigma : [m] \rightarrow [m]$ such that $\widetilde{\mathcal{A}}_i \cong \widetilde{\mathcal{B}}_{\sigma(i)}$ for all $i \in [m]$. By applying the reverse direction of Lemma 3.3 to the individual summands, the permutation equivalence $\widetilde{\mathcal{C}}_A \cong \widetilde{\mathcal{C}}_B$ implies the monomial equivalence $\mathcal{A}_i \cong_{\mathcal{M}} \mathcal{B}_{\sigma(i)}$, for each $i \in [m]$. This proves the claim. $\square$

3.2 Properties of Indecomposable Codes

Lemma 3.6. Let $\mathcal{C} = \bigoplus_{i=1}^m \mathcal{C}_i \subseteq \mathbb{F}_q^n$ be a direct sum of indecomposable codes $\mathcal{C}_i$. For any monomial automorphism $\mu \in \text{MAut}(\mathcal{C})$, there exists a permutation $\sigma \in S_m$ of the summands such that $\mu(\mathcal{C}_i) = \mathcal{C}_{\sigma(i)}$ for all $i \in [m]$.

Here we use $\mu(\mathcal{C}_i)$ to denote the image of μ on the embedded code $\{\mathbf{0}\} + \dots + \mathcal{C}_i + \dots + \{\mathbf{0}\}$, which gives some other embedded code $\{\mathbf{0}\} + \dots + \mathcal{C}_{\sigma(i)} + \dots + \{\mathbf{0}\}$ in the direct sum code $\mathcal{C}$.

Proof Let $\{X_i\}_{i=1}^m$ be supports of the summands $\{\mathcal{C}_i\}_{i=1}^m$ in the index set $X = [n]$ of $\mathcal{C}$; these form a partition $X = \bigsqcup_{i=1}^m X_i$. Fix an arbitrary index $i \in [m]$ and consider any $\mu = (\mathbf{v}_\mu, \pi_\mu) \in \text{MAut}(\mathcal{C})$. We will show that the induced permutation on the index set, $\pi_\mu(X_i)$, is entirely contained within a single coordinate block X_j for some

$j \in [m]$. Since the index sets $\{X_j\}_{j=1}^m$ partition $X = [n]$, the elements of X_i can be partitioned according to their image under the permutation π_μ . Define

$$Y_{i,j} := X_i \cap \pi_\mu^{-1}(X_j) \quad \text{for } j \in [m].$$

Then the collection of non-empty sets among the $\{Y_{i,j}\}_{j=1}^m$ forms a valid partition of X_i .

Suppose, for the sake of contradiction, that the image $\pi_\mu(X_i)$ intersects multiple index sets non-trivially. Then there exist at least two distinct indices, a and b , such that both $Y_{i,a} \neq \emptyset$ and $Y_{i,b} \neq \emptyset$. This gives a partition of X_i into $X_i = Y_{i,a} \sqcup Z$, where $Z := \bigsqcup_{j \neq a} Y_{i,j}$. By Lemma 3.2, since $\mathcal{C}_i$ is an indecomposable code supported on X_i , there exists a bridging element $\mathbf{c} \in \mathcal{C}_i$ with respect to the partition $\{Y_{i,a}, Z\}$. By definition, $\mathbf{c}$ has support in both $Y_{i,a}$ and Z , and it cannot be decomposed into a sum of codewords whose supports are restricted only to $Y_{i,a}$ or Z . Now consider the image of this codeword under μ and denote this by $\mathbf{c}' := \mu(\mathbf{c}) \in \mathcal{C}$. Because $\mathcal{C}$ is a direct sum code, every codeword in $\mathcal{C}$ decomposes uniquely into a sum of components, each restricted to an X_j index set. This allows us to write $\mathbf{c}' = \sum_{j=1}^m \mathbf{u}'_j$, where $\mathbf{u}'_j \in \mathcal{C}_j$ and $\text{supp}(\mathbf{u}'_j) \subseteq X_j$. We apply the inverse automorphism $\mu^{-1} = (\mathbf{v}_\mu^{-1}, \pi_\mu^{-1})$ to this decomposition. The permutation component of μ^{-1} is π_μ^{-1} and its scaling vector is $\mathbf{v}_\mu^{-1} = (v_1^{-1}, \dots, v_n^{-1})$ whose entries are the inverses of the non-zero scalar entries of $\mathbf{v}_\mu$. Applying μ^{-1} to the decomposition of $\mathbf{c}'$, we obtain a decomposition of the original codeword

$$\mathbf{c} = \mu^{-1}(\mathbf{c}') = \sum_{j=1}^m \mu^{-1}(\mathbf{u}'_j).$$

Since $\text{supp}(\mathbf{u}'_j) \subseteq X_j$ and μ induces a bijection, it follows that $\text{supp}(\mu^{-1}(\mathbf{u}'_j)) \subseteq \mu^{-1}(X_j)$. Furthermore, since $\mathbf{c}$ is supported entirely within X_i , each non-zero component $\mu^{-1}(\mathbf{u}'_j)$ must be supported within $X_i \cap \pi_\mu^{-1}(X_j)$, which is precisely the set $Y_{i,j}$. These terms can be grouped according to the partition $\{Y_{i,a}, Z\}$, into

$$\mathbf{c} = \mu^{-1}(\mathbf{u}'_a) + \sum_{j \neq a} \mu^{-1}(\mathbf{u}'_j).$$

Denote the summands by $\mathbf{v}_1 := \mu^{-1}(\mathbf{u}'_a)$ and $\mathbf{v}_2 := \sum_{j \neq a} \mu^{-1}(\mathbf{u}'_j)$. By construction, $\text{supp}(\mathbf{v}_1) \subseteq Y_{i,a}$. Since all other terms map into sets distinct from X_a , their supports are contained in Z , and so $\text{supp}(\mathbf{v}_2) \subseteq Z$. Both $\mathbf{v}_1$ and $\mathbf{v}_2$ are valid codewords in $\mathcal{C}$, which means that $\mathbf{c}$ can be written as a sum $\mathbf{v}_1 + \mathbf{v}_2$ whose summands have disjoint supports across $\{Y_{i,a}, Z\}$. But this contradicts the fact that $\mathbf{c}$ is a bridging element. Thus, exactly one subset $Y_{i,j}$ must be non-empty. This implies that $\pi_\mu(X_i) \subseteq X_j$ for some $j \in [m]$.

Now consider the inverse automorphism $\mu^{-1} \in \text{MAut}(\mathcal{C})$. Since $\pi_\mu(X_i) \subseteq X_j$, we have $X_i \subseteq \pi_\mu^{-1}(X_j)$. Applying the same argument as above to μ^{-1} , we obtain that the set $\pi_\mu^{-1}(X_j)$ is contained entirely in the index set of a single summand. Since $X_i \subseteq \pi_\mu^{-1}(X_j)$, this summand must be X_i , so $\pi_\mu^{-1}(X_j) \subseteq X_i$. Thus, $\pi_\mu^{-1}(X_j) = X_i$, and equivalently, $\pi_\mu(X_i) = X_j$. Therefore, μ induces a permutation $\sigma \in S_m$ on the summands such that $\pi_\mu(X_i) = X_{\sigma(i)}$ for all $i \in [m]$. $\square$

Since $\text{Aut}(\mathcal{C}) \subseteq \text{MAut}(\mathcal{C})$ for any code $\mathcal{C}$, Lemma 3.6 immediately gives the following corollary.

Corollary 3.1. *Let $\mathcal{C} = \bigoplus_{i=1}^m \mathcal{C}_i \subseteq \mathbb{F}_q^n$ be a direct sum of indecomposable codes $\mathcal{C}_i$. For any automorphism $\pi \in \text{Aut}(\mathcal{C})$, there exists a permutation $\sigma \in S_m$ of the summands such that $\pi(\mathcal{C}_i) = \mathcal{C}_{\sigma(i)}$ for all $i \in [m]$.*

Lemma 3.7. *If $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$ are indecomposable codes and $\mathcal{C} := \mathcal{C}_1 \oplus \mathcal{C}_2$, then they satisfy the ratio*

$$\frac{|\text{MAut}(\mathcal{C})|}{|\text{MAut}(\mathcal{C}_1)| \cdot |\text{MAut}(\mathcal{C}_2)|} = \begin{cases} 1 & \text{if } \mathcal{C}_1 \not\cong_{\mathcal{M}} \mathcal{C}_2 \\ 2 & \text{if } \mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2. \end{cases}$$

Proof Let $X = X_1 \sqcup X_2$ be the index set partition induced by the direct sum $\mathcal{C} = \mathcal{C}_1 \oplus \mathcal{C}_2$. Define the direct product of the monomial automorphism groups of $\mathcal{C}_1$ and $\mathcal{C}_2$ by

$$H := \text{MAut}(\mathcal{C}_1) \times \text{MAut}(\mathcal{C}_2) = \{(\mu_1, \mu_2) \mid \mu_i \in \text{MAut}(\mathcal{C}_i), i = 1, 2\} \subseteq \text{MAut}(\mathcal{C}).$$

We consider the two possible cases for equivalence between $\mathcal{C}_1$ and $\mathcal{C}_2$.

($\mathcal{C}_1 \not\cong_{\mathcal{M}} \mathcal{C}_2$) We claim that $H = \text{MAut}(\mathcal{C})$. Assume there exists $\mu \in \text{MAut}(\mathcal{C}) \setminus H$; this has the form $\mu = (\mathbf{v}_\mu, \pi_\mu)$ for some $\mathbf{v}_\mu \in (\mathbb{F}_q^*)^{2n}$ and $\pi_\mu \in S_{2n}$. The associated permutation π_μ must map at least one index from X_1 to X_2 . So, by Lemma 3.6, $\pi_\mu(X_1) = X_2$ which contradicts the assumption. Therefore, $H = \text{MAut}(\mathcal{C})$, and $|\text{MAut}(\mathcal{C})| = |\text{MAut}(\mathcal{C}_1)| \cdot |\text{MAut}(\mathcal{C}_2)|$, so the value of the ratio is 1.

($\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$) By Lemma 3.6, there exists a monomial automorphism $\mu = (\mathbf{v}_\mu, \pi_\mu) \in \text{MAut}(\mathcal{C})$ whose associated permutation π_μ permutes the coordinates of $\mathcal{C}_1$ to $\mathcal{C}_2$ i.e. $\pi_\mu(X_1) = X_2$ and $\pi_\mu(X_2) = X_1$. Then $\mu \notin H$, and

so the cosets H and μH are disjoint. This implies $|\text{MAut}(\mathcal{C})| \geq 2 \cdot |\text{MAut}(\mathcal{C}_1)| \cdot |\text{MAut}(\mathcal{C}_2)|$. Suppose some $\mu' = (\mathbf{v}_{\mu'}, \pi_{\mu'}) \in \text{MAut}(\mathcal{C})$ permutes $\pi_{\mu'}(X_1) = X_2$ and $\pi_{\mu'}(X_2) = X_1$, otherwise $\mu' \in H$. Consider the composition $\kappa := \mu^{-1} \circ \mu'$. The permutation associated with κ acts on X_1 as $\pi_{\kappa}(X_1) = \pi_{\mu^{-1}}(\pi_{\mu'}(X_1)) = \pi_{\mu}^{-1}(X_2) = X_1$. Since κ is a bijection on the finite set X , it must similarly map X_2 to X_2 . Since κ preserves both index sets, we have $\kappa \in H$. Since $\mu^{-1} \circ \mu' = \kappa \in H$, multiplying by μ on the left yields $\mu' = \mu \circ \kappa \in \mu H$.

Therefore, every element of $\text{MAut}(\mathcal{C})$ is either in H or in the left coset μH , establishing $\text{MAut}(\mathcal{C}) = H \sqcup \mu H$. Consequently, $|\text{MAut}(\mathcal{C})| = 2 \cdot |\text{MAut}(\mathcal{C}_1)| \cdot |\text{MAut}(\mathcal{C}_2)|$, and the ratio has value 2. $\square$

Lemma 3.8. *If $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$ are indecomposable codes and $\mathcal{C} := \mathcal{C}_1 \oplus \mathcal{C}_2$, then they satisfy the ratio*

$$\frac{|\text{Aut}(\mathcal{C})|}{|\text{Aut}(\mathcal{C}_1)| \cdot |\text{Aut}(\mathcal{C}_2)|} = \begin{cases} 1 & \text{if } \mathcal{C}_1 \not\cong \mathcal{C}_2 \\ 2 & \text{if } \mathcal{C}_1 \cong \mathcal{C}_2. \end{cases}$$

Proof The proof for the permutation automorphism groups is identical to the proof of Lemma 3.7 for monomial automorphism groups. This is because the effect of scaling in $\text{MAut}(\mathcal{C})$ does not matter, but only the permutation of indices in the index sets. $\square$

Remark 3.1. *A similar result to Lemma 3.8 is given in Equation 34 of [37]. Slepian's formula computes the number of double cosets of any finite group with respect to two of its subgroups. Since no proof or reference is given, and we only require a specific restriction of this ratio that has only two possible evaluations, we present Lemmas 3.7 and 3.8 as independent results.*

Lemma 3.9. *Let $\mathcal{C}$ be a decomposable $[n, k]$ -code such that $\mathcal{C} \cong \bigoplus_{i=1}^m \mathcal{C}_i$ for some $[n_i, k_i]$ -codes $\mathcal{C}_i$. Denote its corresponding index set partition by $X = \bigsqcup_{i=1}^m X_i$, where $X_i \subseteq [n]$ is the index set of $\mathcal{C}_i$ embedded in X . Then, the unique RREF generator matrix $\mathbf{G}_{\text{RREF}}$ of $\mathcal{C}$ has the following properties:*

- (i) *The set of rows of $\mathbf{G}_{\text{RREF}}$ can be partitioned into disjoint sets $R_1, \dots, R_m \subseteq [k]$, such that $|R_i| = k_i$, and every row in R_i has its pivot in X_i .*
- (ii) *For every row vector $\mathbf{r}$ of $\mathbf{G}_{\text{RREF}}$, there exists exactly one index $i \in [m]$ such that $\text{supp}(\mathbf{r}) \subseteq X_i$.*

Proof For simplicity, denote $\mathbf{G} := \mathbf{G}_{\text{RREF}}$. Let $P \subset [n]$ be the set of pivot indices in $\mathbf{G}$; it has size $|P| = k$.

(i) By the definition of a direct sum code, the dimension of $\mathcal{C}$ projected onto each embedded index set X_i is $\dim(\mathcal{C}|_{X_i}) = \dim(\mathcal{C}_i) = k_i$. For every $i \in [m]$, let $\mathbf{G}|_{X_i}$ denote the $k \times n_i$ submatrix of $\mathbf{G}$ comprised of the columns in X_i . This matrix generates the projected code $\mathcal{C}|_{X_i}$, so $\text{rank}(\mathbf{G}|_{X_i}) = \dim(\mathcal{C}|_{X_i}) = k_i$.

Define $R_i := \{\mathbf{r} \in \mathbf{G} \mid \mathbf{r} \text{ has its pivot in a column belonging to } X_i\}$ for every $i \in [m]$. Since the sets X_i are mutually disjoint, the row sets R_i are also disjoint. Because the rows of $\mathbf{G}$ form a basis of $\mathcal{C}$, every row has exactly one pivot, and so $\bigsqcup_{i=1}^m R_i$ partitions the set of rows of $\mathbf{G}$. Denote $k'_i := |R_i|$ for every $i \in [m]$. By the hypothesis $\mathcal{C} \cong \bigoplus_{i=1}^m \mathcal{C}_i$ and the partition of rows of $\mathbf{G}$, we have $\sum_{i=1}^m k'_i = k = \sum_{i=1}^m k_i$. Since there are exactly k'_i rows whose pivots fall in X_i , the rank of the submatrix $\mathbf{G}|_{X_i}$ must be at least $\text{rank}(\mathbf{G}|_{X_i}) = k_i \geq k'_i$. Hence, since $\sum_{i=1}^m k'_i = \sum_{i=1}^m k_i$ and $k_i \geq k'_i$ for all $i \in [m]$, we must strictly have $k'_i = k_i$ for all $i \in [m]$. Therefore, the rows of $\mathbf{G}$ can be partitioned into these sets $R_1, \dots, R_m$.

(ii) Let $\mathbf{r}$ be an arbitrary row vector in $\mathbf{G}$. Since $\mathcal{C} \cong \bigoplus_{i=1}^m \mathcal{C}_i$, the vector $\mathbf{r}$ can be uniquely decomposed as a sum of its projections onto each index set X_i , i.e. $\mathbf{r} = \sum_{i=1}^m \mathbf{v}^{(i)}$, where $\text{supp}(\mathbf{v}^{(i)}) \subseteq X_i$. By the definition of the direct sum, every $\mathbf{v}^{(i)}$ is a valid codeword in $\mathcal{C}$.

We claim that only one of the summands $\mathbf{v}^{(j)}$ is non-zero. By part 1, $\mathbf{r}$ belongs to exactly one set R_j , meaning that its pivot is in some column with index $\ell \in X_j$. Then, by definition of RREF, $\mathbf{r}$ has entry 1 at index ℓ and 0 at every other index in $P \setminus \{\ell\}$.

Take any vector $\mathbf{v}^{(i)}$ where $i \neq j$ and consider the entries of $\mathbf{v}^{(i)}$ in the pivot columns P . By definition, $\text{supp}(\mathbf{v}^{(i)}) \subseteq X_i$ and $X_i \cap X_j = \emptyset$. Then, $\mathbf{v}^{(i)}$ has entry 0 at the pivot index $\ell \in P$ of $\mathbf{r}$, and $\mathbf{v}_{\ell'}^{(i)} = 0$ at any other index $\ell' \in P \setminus \{\ell\}$. Hence, $\mathbf{v}_p^{(i)} = 0$ for all $p \in P$.

Now, since $\mathbf{v}^{(i)} \in \mathcal{C}$, it can be written as a linear combination of the rows of $\mathbf{G}$. By definition of RREF, the submatrix consisting of columns with indices in P forms a $k \times k$ identity matrix. Since $\mathbf{v}_p^{(i)} = 0$ for all $p \in P$ and any linear combination of the rows of $\mathbf{G}$ includes a sum of scalings of the k pivots in P , all scalars in the linear combination must be zero. Hence, $\mathbf{v}^{(i)} = \mathbf{0}$ for all $i \neq j$. Therefore, $\mathbf{r} = \mathbf{v}^{(j)}$ and so $\text{supp}(\mathbf{r}) \subseteq X_j$. $\square$

3.3 How to Decompose a Code

In this section we present a method for efficiently decomposing a linear code into its indecomposable summands. In [37], Slepian presented a method to determine whether a code $\mathcal{C}$ is indecomposable: if its $k \times n$ generator matrix can be put into systematic form, and the columns in the identity block $\mathbf{I}_k$ form a single connected component in the k -vertex graph defined by “connected coordinates”, then the code is indecomposable. This method only indicates *whether* a code is indecomposable. The structural decomposition of linear codes shares a fundamental connection with the notion of connected components in matroid theory in the book [39, Section 8.4]. Building on this connection, Kashyap in [40] showed how algorithmic techniques for matroid decomposition can be explicitly translated to binary linear codes.

We generalize these approaches to obtain an efficient method of decomposing any linear code over an arbitrary finite field $\mathbb{F}_q$. While the foundational idea for this decomposition algorithm has been treated as folklore in several works, including [37, 41], we provide the explicit pseudocode below and formally analyze its complexity.

Our algorithm processes the RREF generator matrix of the input code to obtain a graph representation. By finding all connected components in this graph, the indecomposable summands can then be identified. For the code $\mathcal{C}$ generated by the input matrix, the algorithm outputs an explicit decomposition of $\mathcal{C}$ as a direct sum of some indecomposable codes $\mathcal{C}_1, \dots, \mathcal{C}_m$ up to a permutation of coordinates, along with the permutation π of the columns of the generator matrix that maps $\pi(\mathcal{C}) = \mathcal{C}_1 \oplus \dots \oplus \mathcal{C}_m$. This algorithm will be used as a subroutine in our reductions.

Lemma 3.10. *For any code $\mathcal{C} \subseteq \mathbb{F}_q^n$, Algorithm 1 correctly decomposes it in $O(n^3 \log(q))$ time and finds a set of generators $\{\mathbf{G}_1, \dots, \mathbf{G}_m\}$ for indecomposable codes $\{\mathcal{A}_1, \dots, \mathcal{A}_m\}$ and an associated permutation $\pi \in \mathcal{S}_n$ satisfying $\pi(\mathcal{C}) = \bigoplus_{i=1}^m \mathcal{A}_i$.*

Algorithm 1: Decomposition of a Linear Code

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a $[n, k]$ -code $\mathcal{C}$.

Output: A set generators $\{\mathbf{G}_1, \dots, \mathbf{G}_m\}$ for indecomposable codes $\{\mathcal{A}_1, \dots, \mathcal{A}_m\}$ and a coordinate permutation $\pi \in \mathcal{S}_n$ such that $\pi(\mathcal{C}) = \bigoplus_{i=1}^m \mathcal{A}_i$.

1. Compute the RREF matrix $\mathbf{G}_{\text{RREF}}$ from $\mathbf{G}$.
2. Construct the support graph $\Gamma(\mathcal{C}) = ([n], E)$ from $\mathbf{G}_{\text{RREF}}$.
3. Run Breadth-First Search on $\Gamma(\mathcal{C})$ to identify its m connected components $X_1, \dots, X_m$. Fix any ordering of the elements in each component to get the sequences $Z_1, \dots, Z_m$.
4. Construct a permutation $\pi \in \mathcal{S}_n$ that reorders the indices $[n]$ by mapping them to the concatenated sequence $Z_1, \dots, Z_m$.
5. Define $\mathbf{G}_1, \dots, \mathbf{G}_m$ to be the submatrices of $\mathbf{G}_{\text{RREF}}$ corresponding to the non-zero rows and columns restricted to $X_1, \dots, X_m$, respectively.

return $\{\mathbf{G}_1, \dots, \mathbf{G}_m\}$ and π

Proof Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be the input code of dimension k .

(Correctness) By Construction 2.1, the vertices of the support graph $\Gamma(\mathcal{C})$ represent the coordinate indices of the code $\mathcal{C}$, and the connected components of $\Gamma(\mathcal{C})$ correspond exactly to the disjoint index sets of the indecomposable summands in the decomposition of $\mathcal{C}$. Suppose there are m connected components and let $X_1, \dots, X_m \subset [n]$ denote the elements in these components. These subsets form a partition of the index set $X = [n]$ of $\mathcal{C}$. Running Breadth-First Search on the graph $\Gamma(\mathcal{C})$ will explore all n vertices and identify these m connected components. Then, we fix an (arbitrary) ordering for the elements in each component X_i . Denote the resulting sequence by $Z_i = \{z_{i_1}, z_{i_2}, \dots, z_{i_{n_i}}\}$. Then, we define the permutation $\pi \in \mathcal{S}_n$ by

$$\pi(z_{i_j}) := j + \sum_{\ell=1}^{i-1} n_\ell \quad \text{for } 1 \leq j \leq n_i \text{ and } i \in [m].$$

This permutation shifts the ordered indices of X_1 to the interval $[1, n_1]$, those of X_2 to $[n_1 + 1, n_1 + n_2]$, and so on. By Lemma 3.9, every row vector $\mathbf{r}$ of $\mathbf{G}_{\text{RREF}}$ has its support contained within exactly one of the original index sets X_i . Hence, after applying π to the index set of $\mathcal{C}$, the support of $\mathbf{r}$ is entirely contained within $\pi(X_i)$. By construction of π , the columns of $\mathbf{G}_{\text{RREF}}$ are reordered into disjoint contiguous blocks with index sets $X_1, \dots, X_m$. Thus, the algorithm can identify the block submatrices $\mathbf{G}_1, \dots, \mathbf{G}_m$ of $\mathbf{G}_{\text{RREF}}$ that are supported on $X_1, \dots, X_m$, respectively. By construction and Lemma 3.9, each $\mathbf{G}_i$ is a $k_i \times n_i$ matrix and generates a $[n_i, k_i]$ -code A_i . Therefore, for this permutation π we have $\pi(\mathcal{C}) = \bigoplus_{i=1}^m A_i$ and the algorithm correctly finds π and the corresponding decomposition $\bigoplus_{i=1}^m A_i$.

(Complexity) We analyze the runtime for each step in the algorithm. Each field element in $\mathbb{F}_q$ requires $\log(q)$ bits to represent and $O(\log(q))$ bit operations to read, copy, or compare. Computing the RREF matrix $\mathbf{G}_{\text{RREF}}$ from the $k \times n$ matrix $\mathbf{G}$ can be done using Gaussian elimination in $O(k^2 n \log(q))$ time. Constructing the support graph $\Gamma(\mathcal{C})$ requires iterating through the k rows of $\mathbf{G}_{\text{RREF}}$ and the entries of each row. Each row has at most n non-zero entries in $\mathbb{F}_q^*$, so at most n^2 edges are created per row. Across all k rows, this takes a total of $O(kn^2 \log(q))$ time. Finding the connected components in the graph $\Gamma(\mathcal{C})$ with $|V| = n$ vertices and $|E| \leq n^2$ edges can be done using Breadth-First Search, which takes $O(n + |E|) = O(n^2)$ time. Constructing the permutation π involves concatenating $m \leq n$ disjoint sets, which takes $O(n)$ time. Extracting the blocks $\mathbf{G}_1, \dots, \mathbf{G}_m$ from $\mathbf{G}_{\text{RREF}}$ corresponding to the connected components $X_1, \dots, X_m$ requires memory access operations bounded by the matrix size, and takes $O(kn \log(q))$ time. Since $k \leq n$ and $m \leq n$, the total runtime is bounded by $O(n^3 \log(q))$. $\square$

3.4 Setwise Stabilizer Chains

Now we introduce an algebraic structure called a *setwise stabilizer chain* and prove some key properties. This is adapted from the notion of a *pointwise stabilizer chain* (see Definition 2.11), but is specific to the automorphism group of a code and is defined by the redundancy sets of identical columns in a generator matrix of the code. Similar notions have been used in [10, 41] for graphs.

Setwise stabilizer chains are used in our reductions in Sections 5.3 and 6.3 in the context of iteratively stabilizing columns of a generator matrix of a code, with the goal of finding or counting its automorphisms. The columns of the generator matrix are partitioned into classes of identical columns to efficiently handle trivial permutations without resorting to oracle calls in these reductions.

Definition 3.2 (Setwise Stabilizer Chain). *For any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, let $G := \text{Aut}(\mathcal{C}(\mathbf{G}))$ be the permutation automorphism group of the code generated by $\mathbf{G}$. Denote the column vectors of $\mathbf{G}$ by $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$ and let $B' = \{t_1, \dots, t_s\} \subseteq [n]$ be the sequence of indices of unique column vectors in $\mathbf{G}$ in order of first appearance. This sequence B' defines a partition of the index set $[n]$ into disjoint redundancy classes $E_1, \dots, E_s$, where $E_i := \{c \in [n] \mid \mathbf{g}_c = \mathbf{g}_{t_i}\}$, and let $m_i := |E_i|$. The (permutation) setwise stabilizer chain is the chain of subgroups*

$$G =: G^{(E_0)} \geq G^{(E_1)} \geq \dots \geq G^{(E_s)},$$

where $G^{(E_i)} := \{\pi \in G^{(E_{i-1})} \mid \pi(E_i) = E_i\}$ is the subgroup that stabilizes the set E_i within the preceding subgroup, for each $i \in [s]$.

Similarly, let $M := \text{MAut}(\mathcal{C}(\mathbf{G}))$ be the monomial automorphism group of the code generated by $\mathbf{G}$. Define the redundancy classes $E_1, \dots, E_s$ by $E_i := \{c \in [n] \mid \mathbf{g}_c = \alpha \mathbf{g}_{t_i} \text{ for some } \alpha \in \mathbb{F}_q^*\}$. The corresponding monomial setwise stabilizer chain is the chain of subgroups

$$M =: M^{(E_0)} \geq M^{(E_1)} \geq \dots \geq M^{(E_s)},$$

where $M^{(E_i)} := \{\mu \in M^{(E_{i-1})} \mid \pi_\mu(E_i) = E_i\}$ is the subgroup of monomial automorphisms whose associated permutation $\pi_\mu \in \mathcal{S}_n$ stabilizes the set E_i within the preceding subgroup, for each $i \in [s]$.

Note that the identity map is not the only permutation automorphism in $G = \text{Aut}(\mathcal{C}(\mathbf{G}))$ that stabilizes all indices in the set B' , so this B' is not a base as defined in Definition 2.11. Likewise, the smallest subgroup in the chain, $G^{(E_s)}$, does not necessarily contain only the identity automorphism, as it must for a pointwise stabilizer chain. Instead, $G^{(E_s)}$ is the subgroup generated by column swaps, which is isomorphic to $\prod_{i=1}^s \text{Sym}(E_i)$, the product of the symmetric groups of the equivalence classes.

For the monomial automorphism group M , the smallest subgroup $M^{(E_s)}$ stabilizes $E_1, \dots, E_s$. We refer to these sets as *projective classes*, following the terminology in [2]. If the code $\mathcal{C}(\mathbf{G})$ can be decomposed into c indecomposable summands, the subgroup $M^{(E_s)}$ not only contains automorphisms that internally swap columns within each class E_i , but also automorphisms that multiply each of the c indecomposable summands by the same scalar. We will show in Lemma 3.13 that $M^{(E_s)}$ is isomorphic to $(\mathbb{F}_q^*)^c \times \prod_{i=1}^s \text{Sym}(E_i)$.

For any code $\mathcal{C}$ and setwise stabilizer chain of $G = \text{Aut}(\mathcal{C})$, the number of automorphisms in the group can be found by iterative applications of the Orbit-Stabilizer theorem to consecutive pairs of subgroups along the chain. This is formalized in the following lemma.

Lemma 3.11 (number of permutation automorphisms). *For any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ defining a code $\mathcal{C} = \mathcal{C}(\mathbf{G})$, its automorphism group $G := \text{Aut}(\mathcal{C})$, and a sequence $B' = \{t_1, \dots, t_s\} \subseteq [n]$ of indices of unique column vectors in $\mathbf{G}$ in order of first appearance, the associated setwise stabilizer chain $G \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ has cardinality*

$$|G| = \prod_{i=1}^s |G^{(E_{i-1})} : G^{(E_i)}| \cdot (m_i!).$$

Proof By applying Lagrange's Theorem to consecutive subgroups along the chain, we obtain $|G| = |G^{(E_s)}| \cdot \prod_{i=1}^s |G^{(E_{i-1})} : G^{(E_i)}|$. To prove the claim, it suffices to show that $|G^{(E_s)}| = \prod_{i=1}^s (m_i!)$. By definition, the subgroup $G^{(E_s)}$ stabilizes each E_i setwise, that is, its elements map indices from E_i into E_i , for every $i \in [s]$. Since the sets $E_1, \dots, E_s$ form a partition of the index set $[n]$, every element $\pi \in G^{(E_s)}$ permutes the indices $[n]$ by independently permuting the elements within each E_i . By definition, each equivalence class E_i contains the indices of identical columns in $\mathbf{G}$. Because permuting identical columns leaves $\mathbf{G}$ invariant, every possible permutation of the indices in E_i yields a trivial automorphism of $\mathcal{C}$. Since $E_1, \dots, E_s$ partitions the domain $[n]$ of the automorphisms in $G^{(E_s)}$, there is a bijection between the elements of $G^{(E_s)}$ and the direct product of the symmetric groups of $E_1, \dots, E_s$, i.e. $G^{(E_s)} \cong \text{Sym}(E_1) \times \dots \times \text{Sym}(E_s)$. For each $i \in [s]$, there are $m_i!$ possible permutations for the m_i elements of E_i . Therefore, we obtain $|G^{(E_s)}| = \prod_{i=1}^s (m_i!)$. $\square$

Lemma 3.12. *Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be an indecomposable code. If $\mu = (\mathbf{v}_\mu, \text{id}) \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$ is a monomial automorphism that only scales $\mathcal{C}$, then $\mathbf{v}_\mu = (\lambda, \dots, \lambda)$ for some $\lambda \in \mathbb{F}_q^*$. Consequently, there are exactly $q - 1$ automorphisms in $\text{MAut}(\mathcal{C})$ that scale without permuting codeword indices.*

Proof Let $\mathbf{G} := \mathbf{G}_{\text{RREF}} \in \mathbb{F}_q^{k \times n}$ be the (unique) generator matrix of $\mathcal{C}$ in RREF and denote its entries by $g_{i,j}$ for every row $i \in [k]$ and column $j \in [n]$. Consider the monomial automorphism $\mu = (\mathbf{v}_\mu, \text{id}) \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$, where the permutation is the identity map. By definition, applying μ to the generator matrix $\mathbf{G}$ scales each j -th column of $\mathbf{G}$ by the corresponding j -th entry v_j in $\mathbf{v}_\mu$. Since $\mu \in \text{MAut}(\mathcal{C})$, applying μ to $\mathbf{G}$ must yield a matrix $\mathbf{H} := \mu(\mathbf{G})$ that generates the same code $\mathcal{C} = \mathcal{C}(\mathbf{H})$. We show that this can only happen if the entries of $\mathbf{v}_\mu$ are the same.

Let $\mathbf{D} := \text{diag}(v_1 \dots v_n)$ be the matrix whose diagonal entries are the coordinates of $\mathbf{v}_\mu$. By definition, $\mathbf{H} = \mathbf{G}\mathbf{D}$. Hence, by definition of RREF, we have $\text{RREF}(\mathbf{H}) = \text{RREF}(\mathbf{G}\mathbf{D}) = \mathbf{G}$. Without loss of generality, we can partition $\mathbf{H}$ into $\mathbf{H} = [\mathbf{H}_1 \mid \mathbf{H}_2]$, where $\mathbf{H}_1$ is a full-rank $k \times k$ submatrix and $\mathbf{H}_2$ contains the remaining $n - k$ columns (otherwise the same argument works for the submatrix containing the pivot columns of $\mathbf{H}$). We multiply $\mathbf{H}$ by the inverse matrix $\mathbf{H}_1^{-1}$ to obtain $\text{RREF}(\mathbf{H}) = \mathbf{H}_1^{-1}\mathbf{H} = [\mathbf{I}_k \mid \mathbf{H}_1^{-1}\mathbf{H}_2]$. Every i -th row of $\mathbf{H}$ has pivot column at some index $p_i \in [n]$. By definition of $\mathbf{H}$, the pivot entry $\mathbf{H}_{i,p_i}$ is v_{p_i} . To ensure that the pivot entry is 1 in the RREF matrix $\text{RREF}(\mathbf{H})$, every entry in this row is multiplied by the inverse $v_{p_i}^{-1} \in \mathbb{F}_q^*$. After this scaling, any non-pivot entry in the i -th row and j -th column of $\text{RREF}(\mathbf{H})$ becomes $g_{i,j} \cdot v_j \cdot v_{p_i}^{-1}$. By the equality $\text{RREF}(\mathbf{H}) = \mathbf{G}$, we have $g_{i,j} = g_{i,j} \cdot v_j \cdot v_{p_i}^{-1}$. Hence, for every non-zero entry $g_{i,j} \neq 0$, this implies that $v_j = v_{p_i}$.

Consider the support graph $\Gamma(\mathcal{C})$ of the code $\mathcal{C}$. By Construction 2.1 and the hypothesis that $\mathcal{C}$ is indecomposable, $\Gamma(\mathcal{C})$ is a connected graph, so there is a path between every pair of vertices. By construction, this can only be the case if there are two non-zero entries in every row of $\mathbf{G}$. A path in $\Gamma(\mathcal{C})$ between any pair of vertices $j, j' \in [n]$ corresponds to a sequence of rows in $\mathbf{G}$ with indices $\{i_1, i_2, \dots, i_\ell\}$ in which the i_1 -th row has non-zero entries in column j and some other column j_1 , the i_2 -th row has non-zero entries in column j_1 and some other column j_2 , and so on until the i_ℓ -th row which has non-zero entries in column $j_{\ell-1}$ and column j' .

By the condition above, in every i -th row, both the pivot entry $g_{i,p_i} \neq 0$ and all other entries $g_{i,j} \neq 0$ are multiplied by the same scalar v_j^{-1} . By transitivity and the existence of a path between every pair of vertices, we

obtain that the scalars are all the same, so that $v_1 = v_2 = \dots = v_n$. Therefore, $\mathbf{v}_\mu = (\lambda, \dots, \lambda)$ for some $\lambda \in \mathbb{F}_q^*$. Since there are exactly $q - 1$ choices for λ , there are $q - 1$ scaling-only automorphisms in $\text{MAut}(\mathcal{C})$. $\square$

Lemma 3.13 (number of monomial automorphisms). *For any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ defining a code $\mathcal{C} = \mathcal{C}(\mathbf{G})$ that is decomposable into c indecomposable codes, its monomial automorphism group $M := \text{MAut}(\mathcal{C})$, and a sequence $B' = \{t_1, \dots, t_s\} \subseteq [n]$ of indices of unique projective column representatives in $\mathbf{G}$ in order of first appearance, the associated monomial setwise stabilizer chain $M \geq M^{(E_1)} \geq \dots \geq M^{(E_s)}$ has cardinality*

$$|M| = (q - 1)^c \cdot \prod_{i=1}^s |M^{(E_{i-1})} : M^{(E_i)}| \cdot (m_i!),$$

where $m_i := |E_i|$ for all $i \in [s]$.

Proof By applying Lagrange's Theorem to consecutive subgroups along the chain, we obtain $|M| = |M^{(E_s)}| \cdot \prod_{i=1}^s |M^{(E_{i-1})} : M^{(E_i)}|$. To prove the claim, it suffices to show that $|M^{(E_s)}| = (q - 1)^c \cdot \prod_{i=1}^s (m_i!)$.

By definition, the subgroup $M^{(E_s)}$ stabilizes each projective column class $E_1, \dots, E_s$ setwise. Any monomial automorphism $\mu \in M^{(E_s)} \subset (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$ consists of a permutation automorphism $\pi_\mu \in \mathcal{S}_n$ and a scaling vector $\mathbf{v}_\mu \in (\mathbb{F}_q^*)^n$. We count the number of possible π_μ and $\mathbf{v}_\mu$.

Consider the homomorphism $\rho : M^{(E_s)} \rightarrow \prod_{i=1}^s \text{Sym}(E_i)$ defined by $\rho(\mathbf{v}_\mu, \pi_\mu) = \pi_\mu$; this ignores any scaling. This map ρ is well-defined because every element in $M^{(E_s)}$ stabilizes $E_1, \dots, E_s$ setwise. ρ is also surjective.

The kernel of ρ consists of all scaling-only automorphisms of the form $(\mathbf{v}_\mu, \text{id}) \in M^{(E_s)}$. By hypothesis, $\mathcal{C} \cong_{\mathcal{M}} \mathcal{C}_1 \oplus \dots \oplus \mathcal{C}_c$ for some indecomposable codes $\mathcal{C}_1, \dots, \mathcal{C}_c$. By Lemma 3.12, any scaling-only automorphism in $\text{MAut}(\mathcal{C})$ must scale each indecomposable summand $\mathcal{C}_i$ by a scalar $\lambda_i \in \mathbb{F}_q^*$. Since there are c summands, and the scalar for each summand can be chosen independently, there are $(q - 1)^c$ such scaling-only automorphisms. Hence, $|\ker(\rho)| = (q - 1)^c$.

Any permutation $\pi_\mu \in \prod_{i=1}^s \text{Sym}(E_i)$ can only permute columns within the same projective class. By definition, the columns in each E_i are all scalar multiples of a single representative vector. We can construct a corresponding canonical scaling vector $\mathbf{v}_\mu$ that accounts for the scaling of each column in E_i relative to its representative vector, for every class E_i . For every such canonical vector, there are $(m_i)!$ possible permutations for each class E_i . Hence, $|\text{Im}(\rho)| = \prod_{i=1}^s (m_i!)$.

Then, by the First Isomorphism Theorem, we obtain $|M^{(E_s)}| = |\ker \rho| \cdot |\text{Im}(\rho)| = (q - 1)^c \cdot \prod_{i=1}^s (m_i!)$. $\square$

Definition 3.3 (Strong Generating Set for Setwise Stabilizer Chain). *For any $k, n \in \mathbb{N}$ with $k < n$, and any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ defining a code $\mathcal{C} = \mathcal{C}(\mathbf{G})$, its automorphism group $G' := \text{Aut}(\mathcal{C})$, and a sequence $B' = \{t_1, \dots, t_s\} \subseteq [n]$ of indices of unique column vectors in $\mathbf{G}$ in order of first appearance, let $G' \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ be the associated setwise stabilizer chain. A setwise strong generating set for G' with respect to B' is a set S' that generates $\langle S' \rangle = G'$ and satisfies the property $\langle S' \cap G^{(E_i)} \rangle = G^{(E_i)}$ for all $i \in [s]$.*

Remark 3.2. *For any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, let $G := \text{Aut}(\mathcal{C}(\mathbf{G}))$ and $B' = \{t_1, \dots, t_s\} \subseteq [n]$ be the sequence of indices of unique column vectors in $\mathbf{G}$ in order of first appearance. A strong generating set S for G with respect to base $B = [n]$ can be obtained from any strong generating set S' for the setwise stabilizer chain $G = G^{(E_0)} \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ defined by B' (see [36, Section 4.2]).*

Lemma 3.14. *For any matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, let $G := \text{Aut}(\mathcal{C}(\mathbf{G}))$ and $B' = \{t_1, \dots, t_s\} \subseteq [n]$ be the sequence of indices of unique column vectors in $\mathbf{G}$ in order of first appearance. Let $G = G^{(E_0)} \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ be the setwise stabilizer chain defined by B' . Suppose that S_{E_s} is a generating set for $G^{(E_s)}$, and R_i is a set of left coset representatives for the quotient space $G^{(E_{i-1})}/G^{(E_i)}$, for each $i \in [s]$. Then, the union of these sets generates the full group group*

$$G = \left\langle S_{E_s} \cup \bigcup_{i=1}^s R_i \right\rangle.$$

Proof We use induction on the subgroups of the chain, from smallest to largest. For the base case $i = s$, we have $G^{(E_s)} = \langle S_{E_s} \rangle$ by definition. Suppose that the subgroup $G^{(E_i)}$ is generated by $S_{E_s} \cup \bigcup_{j=i+1}^s R_j$ for some $i \in [s]$. By applying [Lemma 2.7](#) to the adjacent subgroups $G^{(E_i)} \leq G^{(E_{i-1})}$, we obtain that $G^{(E_{i-1})}$ is generated by the union of the generating set for $G^{(E_i)}$ and the coset representatives R_i . Iterating this application of [Lemma 2.7](#) for every step $i = s, s-1, \dots, 1$, we obtain that the full group $G^{(E_0)} = G$ is generated by the union of S_{E_s} and all transversals R_i . $\square$

4 APART and MAPART

In this section we prove that determining whether two codes are equivalent is a polynomially equivalent problem to computing the partition of a code's coordinates into orbits under its automorphism group. We first state the main results.

Theorem 4.1 (PCE reduces to APART, LCE reduces to MAPART). *There is a reduction from PCE (resp. LCE) on any pair of n -length codes to APART (resp. MAPART) that runs in $O(n^3 \log(q))$ time and makes at most n^2 oracle calls.*

Theorem 4.2 (APART reduces to PCE, MAPART reduces to LCE). *There is a reduction from APART (resp. MAPART) on any n -length code to PCE (resp. LCE) which runs in $O(n^3 \log(q))$ time and makes $O(n^2)$ oracle calls on codes of length at most $2n$.*

We prove the reduction from PCE in [Section 4.1](#) and its monomial analog in [Section 4.2](#). We prove the reverse reductions in [Sections 4.3](#) and [4.4](#).

4.1 PCE Reduces to APART

Proposition 4.1 (PCE Reduces to APART). *[Algorithm 2](#) solves PCE for any two n -length q -ary codes in $O(n^3 \log(q))$ time using at most n^2 calls to an APART oracle.*

Proof Let $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$ be the two input codes.

(Correctness) In Step 1, the algorithm calls [Algorithm 1](#) to obtain a decomposition of the input codes $\mathcal{C}_1$ and $\mathcal{C}_2$ into indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$, respectively. By [Lemma 3.1](#), any code can be decomposed uniquely into a direct sum of indecomposable codes, up to equivalence and permutation of the summands. Hence, $\mathcal{C}_1 \cong \mathcal{C}_2$ only if $\mathcal{C}_1$ and $\mathcal{C}_2$ have the same number of summands, i.e. $m_1 = m = m_2$. In this case, there exists a bijection that pairs each summand A_i of $\mathcal{C}_1$ with a summand B_j of $\mathcal{C}_2$.

In Step 2, the algorithm constructs a bipartite graph with vertices $U = \{a_1, \dots, a_m\}$ and $V = \{b_1, \dots, b_m\}$ representing the summands of $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively. We claim that the edges between the vertex sets represent equivalence between the corresponding summands. Consider an arbitrary pair of vertices a_i and b_j for some $i, j \in [m]$. There are two cases to consider for their respective summands A_i and B_j .

Suppose that $A_i \cong B_j$. In this case, A_i and B_j must have the same length n_i and there exists an isomorphism $\pi \in S_{n_i}$ that maps $\pi(A_i) = B_j$. Any such permutation π yields an automorphism $\alpha_\pi \in S_{2n_i}$ of the direct sum code $A_i \oplus B_j \subseteq \mathbb{F}_q^{2n}$. This α_π acts on the index set by swapping the summands; this has the form $\alpha_\pi(A_i \oplus B_j) = \pi^{-1}(B_j) \oplus \pi(A_i)$. This means that there exists an orbit that crosses the boundary between the embedded index sets of A_i and B_j and contains an index from $\{1, \dots, n_i\}$ and another from $\{n_i + 1, \dots, 2n_i\}$. The APART oracle will find this orbit and add an edge (a_i, b_j) to represent the equivalence between A_i and B_j .

Conversely, suppose that $A_i \not\cong B_j$. Then the direct sum $A_i \oplus B_j$ forms a decomposition consisting of two non-equivalent indecomposable codes. By [Corollary 3.1](#), any automorphism of a direct sum of indecomposable codes must map each summand to an equivalent summand. Hence, no automorphism can map any coordinate of A_i into the index set of B_j . Thus, no orbit returned by APART will contain indices from the coordinates sets of both A_i and B_j , so no edge (a_i, b_j) will be added. Therefore, there is exactly one edge in E between every pair of equivalent summands.

In Step 3, the Hopcroft-Karp algorithm finds a maximum bipartite matching [\[42\]](#). By [Lemma 3.1](#), $\mathcal{C}_1 \cong \mathcal{C}_2$, if and only if there is a bijection between the m summands in the codes' decomposition. If $\mathcal{C}_1 \cong \mathcal{C}_2$, this corresponds to a perfect matching of size m in the constructed graph, and so the algorithm will output YES. Otherwise, if $\mathcal{C}_1 \not\cong \mathcal{C}_2$, the size of the maximum matching will be less than m , and so the algorithm will output No.

Algorithm 2: Reduction from PCE to APART

Input : Generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: YES if $\mathcal{C}_1 \cong \mathcal{C}_2$, NO otherwise.

// Step 1: Code decomposition.

Run [Algorithm 1](#) on $\mathbf{G}_1, \mathbf{G}_2$ to obtain decompositions $\pi_1(\mathcal{C}_1) = \bigoplus_{i=1}^{m_1} A_i$ and $\pi_2(\mathcal{C}_2) = \bigoplus_{j=1}^{m_2} B_j$,
for some indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$ and permutations $\pi_1, \pi_2 \in \mathcal{S}_n$.

if $m_1 \neq m_2$ **then**

return NO.

Let $m := m_1 = m_2$.

// Step 2: Construction of bipartite equivalence graph.

Define vertex sets $U := \{a_1, \dots, a_m\}$ and $V := \{b_1, \dots, b_m\}$.

Set $E := \emptyset$.

for $i \leftarrow 1$ **to** m **do**

$n_i :=$ length of code A_i .

for $j \leftarrow 1$ **to** m **do**

 Run APART($A_i \oplus B_j$) to obtain a set of coordinate orbits $\mathcal{O}$.

 // Find an orbit containing indices from both coord sets of A_i and B_j .

if $\exists O \in \mathcal{O}$ with $O \cap [n_i] \neq \emptyset \neq O \setminus [n_i]$ **then**

$E \leftarrow E \cup \{(a_i, b_j)\}$.

Construct graph $G = (U \sqcup V, E)$.

// Step 3: Find a perfect matching between summands.

Run the HOPCROFT-KARP ALGORITHM on G to obtain a maximum bipartite matching $M \subseteq E$

if $|M| = m$ **then**

return YES.

return NO.

(Complexity) The number of indecomposable components for each input code is bounded by $m_i \leq n$. If the number of summands in the decompositions match, then Step 2 will test every pair of summands from different code decompositions. This requires $m^2 \leq n^2$ calls to the APART oracle.

Decomposing the codes in Step 1 uses [Algorithm 1](#), which requires $O(n^3)$ time. For each test pair (A_i, B_j) where $i, j \in [m]$, constructing the direct sum $A_i \oplus B_j$ takes $O(n_i \log(q))$ time, requiring a total of $O(n \log(q))$ time across all summands. Finally, the Hopcroft-Karp algorithm is used to find a maximum bipartite matching in the graph with $|V| = 2m \leq 2n$ vertices and $|E| \leq m^2 \leq n^2$ edges; this requires $O(|E| \sqrt{|V|}) = O(n^{2.5})$ time. Thus, the total runtime is $O(n^3) + O(n \log(q)) + O(n^{2.5}) = O(n^3 \log(q))$ time. $\square$

4.2 LCE Reduces to MAPART

Now we prove the monomial analog of [Proposition 4.1](#). The reduction algorithm is nearly identical to [Algorithm 2](#) but uses the properties of monomial equivalence and a MAPART oracle instead of APART. For completeness, we include the full algorithm and proof below.

Proposition 4.2 (LCE Reduces to MAPART). *[Algorithm 3](#) solves LCE for any two n -length q -ary codes in $O(n^3 \log(q))$ time using at most n^2 calls to a MAPART oracle.*

Proof The proof is identical to that of [Algorithm 2](#), but replacing [Lemma 3.1](#) and [Corollary 3.1](#) with the analogous results in [Lemmas 3.5](#) and [3.6](#). $\square$

Algorithm 3: Reduction from LCE to MAPART

Input : Generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: YES if $\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$, NO otherwise.

// Step 1: Code decomposition.

Run [Algorithm 1](#) on $\mathbf{G}_1, \mathbf{G}_2$ to obtain decompositions $\pi_1(\mathcal{C}_1) = \bigoplus_{i=1}^{m_1} A_i$ and $\pi_2(\mathcal{C}_2) = \bigoplus_{j=1}^{m_2} B_j$,
for some indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$ and permutations $\pi_1, \pi_2 \in \mathcal{S}_n$.

if $m_1 \neq m_2$ then

└ return NO.

Let $m := m_1 = m_2$.

// Step 2: Construction of bipartite equivalence graph.

Define vertex sets $U := \{a_1, \dots, a_m\}$ and $V := \{b_1, \dots, b_m\}$.

Set $E := \emptyset$.

for $i \leftarrow 1$ to m do

└ $n_i :=$ length of code A_i .

└ for $j \leftarrow 1$ to m do

└ └ Run MAPART($A_i \oplus B_j$) to obtain a set of monomial coordinate orbits $\mathcal{O}$.

└ └ // Find an orbit containing indices from coord sets of both A_i and B_j .

└ └ if $\exists O \in \mathcal{O}$ with $O \cap [n_i] \neq \emptyset \neq O \setminus [n_i]$ then

└ └ └ $E \leftarrow E \cup \{(a_i, b_j)\}$.

Construct graph $G = (U \sqcup V, E)$.

// Step 3: Find a perfect matching between summands.

Run the HOPCROFT-KARP ALGORITHM on G to obtain a maximum bipartite matching $M \subseteq E$

if $|M| = m$ then

└ return YES.

return NO.

4.3 APART Reduces to PCE

The reduction from APART to PCE uses a similar approach to that used in the search-to-decision PCE reduction from [\[1\]](#). In this algorithm, we iterate through the index set $[n]$ of the input code and use the PCE oracle to check whether each index $i \in [n]$ can be mapped to any other index. If so, we assign both i and its image to the same orbit, and in this way partition the entire index set $[n]$ into orbits.

Let m denote the maximum number of times a column appears in the generator matrix $\mathbf{G}$ of the input code $\mathcal{C}$. Denote the columns of $\mathbf{G}$ by $\mathbf{g}_1, \dots, \mathbf{g}_n$ and let $\mathbf{G}_i$ be the matrix consisting of exactly m copies of column $\mathbf{g}_i$.

Proposition 4.3. *Algorithm 4 correctly solves APART for any n -length q -ary code in $O(n^3 \log(q))$ time using $O(n^2)$ calls to a PCE oracle on codes of length at most $2n$.*

Proof Let $\mathcal{C} \subseteq \mathbb{F}_q^n$ be the input code and $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ be its generator matrix.

(Correctness) We show that [Algorithm 4](#) partitions the index set $[n]$ into disjoint orbits $X_1, \dots, X_\ell$ such that $i, j \in X_t$ (for $t \in [\ell]$) if and only if there exists an automorphism $\alpha \in \text{Aut}(\mathcal{C})$ for which $\alpha(i) = j$.

The algorithm maintains a set of unassigned indices U to ensure that each index is assigned to exactly one orbit. This guarantees that the output collection of subsets forms a partition of $[n]$. In each iteration of the while-loop, the smallest unassigned index i is used as a base for a new orbit set. Then the algorithm iterates through all remaining unassigned indices $j \in U$ to determine if j belongs in the orbit of i .

To do this, the algorithm constructs the augmented matrices $\mathbf{G}_A = [\mathbf{G} \mid \mathbf{G}_i]$ and $\mathbf{G}_B = [\mathbf{G} \mid \mathbf{G}_j]$ by appending exactly m identical copies of column $\mathbf{g}_i$ and $\mathbf{g}_j$, respectively, where m is the maximum number of times a column appears in the given generator matrix $\mathbf{G}$. By definition, appending m copies guarantees that the frequencies of $\mathbf{g}_i$ in $\mathbf{G}_A$ and $\mathbf{g}_j$ in $\mathbf{G}_B$ become at least $m + 1$. Since no other column in the base matrix appears more than m

Algorithm 4: Reduction from APART to PCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a code $\mathcal{C} \subseteq \mathbb{F}_q^n$.

Output: The set of coordinate orbits $\{X_1, \dots, X_\ell\}$ that partition $[n]$.

```
U := {1, 2, ..., n}. // set of unassigned indices
X1, ..., Xn ← ∅. // orbits of indices
m := maximum column multiplicity in G.

while U ≠ ∅ do
    i ← min{U}. // smallest unassigned index
    Xi ← Xi ∪ {i}. // maps to itself under identity automorphism
    U ← U \ {i}. // mark i as assigned
    for j ∈ U do
        GA ← [G | Gi]. // append m copies of column i
        GB ← [G | Gj]. // append m copies of column j
        // Test if j is in the orbit of i using the PCE oracle.
        if PCE(GA, GB) = YES then
            Xi ← Xi ∪ {j}. // add j to orbit of i
            U ← U \ {j}. // mark j as assigned
    end for
end while

ℓ := number of non-empty sets Xi.

return X1, ..., Xℓ.
```

times in $\mathbf{G}_A$ and $\mathbf{G}_B$, the columns in the appended blocks $\mathbf{G}_i$ and $\mathbf{G}_j$ have a unique frequency. By [Lemma 2.1](#), any permutation π for which $\pi(\mathcal{C}(\mathbf{G}_A)) = \mathcal{C}(\mathbf{G}_B)$ will necessarily map all copies of $\mathbf{g}_i$ to all copies of $\mathbf{g}_j$. Hence, the PCE oracle outputs YES on input $(\mathbf{G}_A, \mathbf{G}_B)$ if and only if there exists an automorphism of the original code $\mathcal{C}$ that maps index i to j . In this way, the algorithm correctly identifies all j in the orbit of i to define their orbit set. In the case where every index maps to itself, there will be n singleton orbits, so up to n of the sets $X_1, \dots, X_n$ will be populated. The algorithm outputs the non-empty sets among these as the orbit partition.

(Complexity) First we bound the number of oracle calls. The while-loop iteratively partitions the n indices into orbits by testing a base index i against all remaining unassigned indices in U . In the worst case, where the automorphism group is trivial and all orbits have size 1, exhaustively testing each index i against all remaining $n - i$ targets requires up to $\sum_{i=1}^{n-1} (n - i) = n(n - 1)/2 = O(n^2)$ PCE oracle calls.

Now we bound the runtime. For each test, constructing $\mathbf{G}_A$ and $\mathbf{G}_B$ requires appending $m \leq n$ columns to $\mathbf{G}$, yielding test matrices of width at most $2n$ over the q -ary field $\mathbb{F}_q$. This augmentation, along with the basic set operations for U and X_i , takes $O(n \log(q))$ time per query. Therefore, [Algorithm 4](#) solves APART for any n -length q -ary code in $O(n^3 \log(q))$ time. $\square$

4.4 MAPART Reduces to LCE

Now we prove the monomial analog of [Proposition 4.3](#). The reduction algorithm is nearly identical to [Algorithm 4](#) but uses the properties of monomial equivalence and an LCE oracle instead of PCE. For completeness, we include the full algorithm and proof below.

Let m denote the maximum number of times any column, or any of its non-zero scalar multiples, appears in the generator matrix $\mathbf{G}$ of the input code $\mathcal{C}$; this is called the *maximum projective multiplicity* of $\mathbf{G}$. Denote the columns of $\mathbf{G}$ by $\mathbf{g}_1, \dots, \mathbf{g}_n$ and define $\mathbf{G}_i$ to be the matrix consisting of exactly m copies of $\mathbf{g}_i$.

Proposition 4.4. *Algorithm 5 correctly solves MAPART for any n -length q -ary code in $O(n^3 \log(q))$ time using $O(n^2)$ calls to an LCE oracle on codes of length at most $2n$.*

Proof The proof is identical to that of [Algorithm 4](#), but replacing [Lemma 2.1](#) with the analogous result in [Lemma 2.2](#). $\square$

Algorithm 5: Reduction from MAPART to LCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a code $\mathcal{C} \subseteq \mathbb{F}_q^n$.

Output: The set of monomial index orbits $\{X_1, \dots, X_\ell\}$ that partition $[n]$.

$U := \{1, 2, \dots, n\}$. // set of unassigned indices

$X_1, \dots, X_n \leftarrow \emptyset$. // monomial index orbits

$m :=$ maximum projective column multiplicity in $\mathbf{G}$.

while $U \neq \emptyset$ **do**

$i \leftarrow \min\{U\}$. // smallest unassigned coord

$X_i \leftarrow X_i \cup \{i\}$. // maps to itself under identity

$U \leftarrow U \setminus \{i\}$. // mark i as assigned

for $j \in U$ **do**

$\mathbf{G}_A \leftarrow [\mathbf{G} \mid \mathbf{G}_i]$. // append m copies of column i

$\mathbf{G}_B \leftarrow [\mathbf{G} \mid \mathbf{G}_j]$. // append m copies of column j

 // Test if j is in the monomial orbit of i using the LCE oracle.

if $\text{LCE}(\mathbf{G}_A, \mathbf{G}_B) = \text{YES}$ **then**

$X_i \leftarrow X_i \cup \{j\}$. // add j to spatial orbit of i

$U \leftarrow U \setminus \{j\}$. // mark j as assigned

$\ell :=$ number of non-empty sets X_i .

return $X_1, \dots, X_\ell$.

5 ACOUNT and MACOUNT

In this section, we prove that determining whether two codes are equivalent is a polynomially equivalent problem to computing the cardinalities of their automorphism groups.

Theorem 5.1 (PCE reduces to ACOUNT, LCE reduces to MACOUNT). *There is a reduction from PCE (resp. LCE) on any pair of n -length codes to ACOUNT (resp. MACOUNT) which runs in $O(n^3 \log(q))$ time and makes at most $3n^2$ oracle calls.*

Theorem 5.2 (ACOUNT reduces to PCE, MACOUNT reduces to LCE). *There is a reduction from ACOUNT (resp. MACOUNT) on any n -length q -ary code to PCE (resp. LCE) which runs in $O(n^6 \log(q))$ time and makes at most $O(n^2)$ oracle calls.*

We prove the reduction from PCE in [Section 5.1](#) and its monomial analog in [Section 5.2](#). We prove the reverse reductions in [Sections 5.3](#) and [5.4](#).

5.1 PCE Reduces to ACOUNT

The reduction from PCE takes the input codes and decomposes them into direct sums of indecomposable codes. These can only be permutation equivalent if the decompositions have the same number of summands and the summands can be paired into isomorphic pairs of codes. The algorithm identifies pairs of isomorphic summands using a ratio test, as given by [Lemma 3.8](#).

Proposition 5.1. *[Algorithm 6](#) correctly solves PCE for any pair of input n -length codes in $O(n^3 \log(q))$ time using at most $3n^2$ calls to an ACOUNT oracle.*

Proof Let $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$ be the two input codes

(*Correctness*) To prove correctness, we must show that the algorithm outputs YES if given a pair of equivalent codes, and outputs NO otherwise. Consider the two possible cases.

Algorithm 6: Reduction from PCE to ACOUNT

Input : Codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: YES if $\mathcal{C}_1 \cong \mathcal{C}_2$, NO otherwise.

// Step 1: Code decomposition.

Run [Algorithm 1](#) on each input code to obtain the decompositions $\mathcal{C}_1 \cong \bigoplus_{i=1}^m A_i$ and

$$\mathcal{C}_2 \cong \bigoplus_{j=1}^{m'} B_j,$$

for some $[n_{A_i}, k_{A_i}]$ -codes $\{A_i\}_{i=1}^m$ and $[n_{B_j}, k_{B_j}]$ -codes $\{B_j\}_{j=1}^{m'}$.

// Step 2: Match summands between $\mathcal{C}_1$ and $\mathcal{C}_2$.

if $m \neq m'$ **then**

return NO.

end

$I \leftarrow \{1, 2, \dots, m\}$

for $i \leftarrow 1$ **to** m **do**

 match-found $\leftarrow$ NO

for each $j \in I$ **do**

 // Test for isomorphism using the ACOUNT ratio test.

 Run ACOUNT oracle to obtain the cardinalities $|\text{Aut}(A_i \oplus B_j)|, |\text{Aut}(A_i)|, |\text{Aut}(B_j)|$.

 Compute $R := |\text{Aut}(A_i \oplus B_j)| / (|\text{Aut}(A_i)| \cdot |\text{Aut}(B_j)|)$.

if $R = 2$ **then**

$I \leftarrow I \setminus \{j\}$

 match-found $\leftarrow$ YES

break.

end

end

if match-found = NO **then**

return NO

end

end

return YES

($\mathcal{C}_1 \cong \mathcal{C}_2$): By [Lemma 3.1](#), the decompositions of $\mathcal{C}_1$ and $\mathcal{C}_2$ found in Step 1 are unique up to permutation equivalence. Since the codes are permutation equivalent, we must have $m = m'$ and $A_i \cong B_{\sigma(i)}$ for all $i \in [m]$ for some permutation σ on $[m]$. Step 2 of the algorithm will implicitly match the summands according to this unknown σ . Since $m = m'$, the algorithm bypasses the first check and implements the nested for-loop to test isomorphism for each (non-paired) pair of summands, A_i and B_j . In this loop, the ACOUNT oracle is called three times to obtain the relevant cardinalities and their ratio is computed. By [Lemma 3.8](#), the value of this ratio correctly indicates whether the pair of summands is equivalent: the value is 2 if and only if the two summands form an equivalent pair. If multisets $\{A_1, \dots, A_m\}$ and $\{B_1, \dots, B_m\}$ are paired perfectly in this way, then match-found = YES when the algorithm terminates. Since the nested for-loop exhaustively searches for a valid matching, and we know such a matching σ exists, the algorithm will output YES.

($\mathcal{C}_1 \not\cong \mathcal{C}_2$): By [Lemma 3.1](#), the decompositions of $\mathcal{C}_1$ and $\mathcal{C}_2$ found in Step 1 are unique up to permutation equivalence. Suppose for contradiction that the algorithm outputs YES in this case. Then, it must have found a matching μ between the summands in the multisets $\{A_1, \dots, A_m\}$ and $\{B_1, \dots, B_{m'}\}$ such that $A_i \cong B_{\mu(i)}$ for all $i \in [m]$. But this would imply that $m = m'$ and

$$\mathcal{C}_1 \cong A_1 \oplus \dots \oplus A_m \cong B_1 \oplus \dots \oplus B_m \cong \mathcal{C}_2,$$

which contradicts the assumption that the codes are not permutationally equivalent. Therefore, the algorithm must return NO in this case.

(Complexity) By [Lemma 3.10](#), decomposing both n -length q -ary input codes takes $2 \cdot O(n^3 \log(q)) = O(n^3 \log(q))$ time. The matching process in Step 2 involves a nested for-loop, where the outer loop iterates over the m summands, and the inner loop iterates over the remaining $\leq m$ unmatched summands. In the worst case, this requires $m + (m-1) + \dots + 1 = m(m+1)/2$ iterations. The ACOUNT oracle is called at most three times per iteration, so at most $3 \cdot m(m+1)/2 \leq 3m^2 \leq 3n^2$ oracle calls are made in total. Constructing the direct sum code in each

iteration requires $O((n_{A_i} + n_{B_j}) \log(q)) = O(n \log(q))$ time, amounting to a total of $O(m^2 n \log(q)) = O(n^3 \log(q))$ time across the entire algorithm. Therefore, the total runtime is bounded by $O(n^3 \log(q))$. $\square$

5.2 LCE Reduces to MACOUNT

Now we prove the monomial analog of [Proposition 5.1](#). The reduction algorithm is nearly identical to [Algorithm 6](#) but uses the properties of monomial equivalence and a MACOUNT oracle instead of ACOUNT. For completeness, we include the algorithm and its proof here.

Proposition 5.2. *Algorithm 7 correctly solves LCE for any pair of input n -length q -ary codes in $O(n^3 \log(q))$ time, using at most $3n^2$ calls to an MACOUNT oracle.*

Algorithm 7: Reduction from LCE to MACOUNT

Input : Codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: YES if $\mathcal{C}_1 \cong_{\mathcal{M}} \mathcal{C}_2$, NO otherwise.

// Step 1: Code decomposition.

Run [Algorithm 1](#) on each input code to obtain the decompositions $\mathcal{C}_1 \cong \bigoplus_{i=1}^m A_i$ and

$\mathcal{C}_2 \cong \bigoplus_{j=1}^{m'} B_j$,

for some $[n_{A_i}, k_{A_i}]$ -codes $\{A_i\}_{i=1}^m$ and $[n_{B_j}, k_{B_j}]$ -codes $\{B_j\}_{j=1}^{m'}$.

// Step 2: Match summands between $\mathcal{C}_1$ and $\mathcal{C}_2$.

if $m \neq m'$ then

 | return NO.

end

$I \leftarrow \{1, 2, \dots, m\}$

for $i \leftarrow 1$ to m do

 match-found $\leftarrow$ NO.

 for each $j \in I$ do

 // Test for isomorphism using the MACOUNT ratio test.

 Run MACOUNT oracle to obtain the cardinalities

$|\text{MAut}(A_i \oplus B_j)|, |\text{MAut}(A_i)|, |\text{MAut}(B_j)|$.

 Compute $R := |\text{MAut}(A_i \oplus B_j)| / (|\text{MAut}(A_i)| \cdot |\text{MAut}(B_j)|)$.

 if $R = 2$ then

$I \leftarrow I \setminus \{j\}$

 match-found $\leftarrow$ YES

 break

 end

 end

 if match-found = NO then

 | return NO

 end

end

return YES

Proof The proof is identical to that of [Proposition 5.1](#), but replacing [Lemmas 3.1](#) and [3.8](#) with the analogous results in [Lemmas 3.5](#) and [3.7](#). $\square$

5.3 ACOUNT to PCE

For the reduction from ACOUNT to PCE, we use a setwise stabilizer chain for the permutation automorphism group of the input code, as defined in [Definition 3.2](#). For any code $\mathcal{C}$ with generator matrix $\mathbf{G}$, let $G := \text{Aut}(\mathcal{C})$ denote its permutation automorphism group and let $G = G^{(E_0)} \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ be the setwise stabilizer chain given by $\mathbf{G}$. To compute the cardinality $|G|$, we use the expression

from [Lemma 3.11](#), which states

$$|G| = \prod_{i=1}^s |G^{(E_{i-1})} : G^{(E_i)}| \cdot (m_i!).$$

To compute $|G^{(E_{i-1})} : G^{(E_i)}|$ for every $1 \leq i \leq s$, we count the number automorphisms that map indices in E_i to some other E_j under the condition that all indices in $E_0, \dots, E_{i-1}$ are mapped back into their respective sets. To count all such automorphisms, we iterate through every index j in the orbit of i and test whether there is an automorphism that maps i to j using the PCE oracle. This technique loosely resembles that used in the search-to-decision PCE reduction from [\[1\]](#), which uses a frequency argument to determine the existence of a permutation map between pairs of indices. The key difference in our algorithm is that all previous indices are fixed and mapped to themselves in each iteration. The algorithm relies on the crucial fact that two generator matrices can generate equivalent codes only if they have the same frequency distribution of columns.

Proposition 5.3. *Algorithm 8 correctly solves ACOUNT for any q -ary n -length code in $O(n^6 \log(q))$ time using at most $O(n^2)$ calls to a PCE oracle.*

Proof Let $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ be the input generator matrix of the code $\mathcal{C}$, and $G = \text{Aut}(\mathcal{C})$ denote its automorphism group. Let $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$ denote the columns of $\mathbf{G}$ in order of appearance. We prove that [Algorithm 8](#) correctly outputs $|G|$ by traversing a setwise stabilizer chain for G and applying [Lemma 3.11](#).

(Correctness) The algorithm first partitions the columns of $\mathbf{G}$ into disjoint redundancy classes $\mathcal{E} = \{E_1, \dots, E_s\}$, where each E_i contains identical columns of a particular type, and no two classes contain the same vector. For this sorting process, the algorithm identifies the indices of the unique column vectors in $\mathbf{G}$ in order of first appearance; let $B' = \{t_1, \dots, t_s\} \subseteq [n]$ denote the elements of this subset. Note that each index $t_i \in B'$ defines the redundancy class $E_i \in \mathcal{E}$. By [Definition 3.2](#), this B' defines a setwise stabilizer chain $G \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$.

In the next for-loop, the algorithm initializes the group order counter $N = \prod_{E \in \mathcal{E}} |E|!$. By [Lemma 3.11](#), this value corresponds to the order of the smallest subgroup $G^{(E_s)}$ in the chain. It represents the core of automorphisms that permute identical columns within their respective classes in $\mathcal{E}$.

Next, the algorithm makes a single oracle call to an APART oracle to obtain a partition of the index set $[n]$ into orbits under automorphisms in G . While not strictly necessary, this step provides an optimization that restricts the search space for automorphisms in the last step.

To compute the remaining number of automorphisms in G , the algorithm traverses the chain $G \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ in descending order by iterating through the elements of B' . In the t_i -th iteration, we compute the quotient index $|G^{(E_{i-1})} : G^{(E_i)}|$ by setwise stabilizing the columns in E_{i-1} and searching for automorphisms that map E_i to some E_j for all larger indices $t_j \in B'$ in the orbit X of t_i . By testing only the unique representatives of each redundancy class, the algorithm evaluates exactly one target per class. This prevents any redundant counting for identical columns within a target class, as the permutations among columns of a class are already counted by the initial $N = \prod_{i=1}^s |E_i|!$.

Any automorphism that maps E_i to E_j maps all copies of column $\mathbf{g}_{t_i}$ to all copies of column $\mathbf{g}_{t_j}$, where $t_i, t_j \in B'$. To test for the existence of such automorphisms, we use a similar technique to that in [\[1, 4\]](#) and augment the matrix $\mathbf{G}$ with copies of columns to ensure unique frequency. In particular, we construct test matrices $\mathbf{G}_A$ and $\mathbf{G}_B$ by appending blocks of $(l+1)m$ copies of $\mathbf{g}_{t_i}$ and $(l+1)m$ copies of $\mathbf{g}_{t_j}$, respectively, to the current stabilized matrix in which column classes $E_1, \dots, E_{i-1}$ are fixed setwise. By definition of m and l , no other column in $\mathbf{G}_A$ and $\mathbf{G}_B$ appears the same number of times, so $\mathbf{g}_{t_i}$ and $\mathbf{g}_{t_j}$ have the same unique frequency. By [Lemma 2.1](#), the PCE oracle can only respond YES if there is an automorphism that maps E_i to E_j . Thus, the oracle is forced to decide equivalence exclusively within the stabilizer subgroup $G^{(E_{i-1})}$. The counter d_i increments for each target class found, beginning from 1 which counts the trivial map of E_i to itself. Hence, once the inner for-loop is completed, d_i represents the setwise orbit size, i.e. $d_i = |G^{(E_{i-1})} : G^{(E_i)}|$.

At the end of each iteration $t_i \in B'$, the running total count N is multiplied by d_i . Because the algorithm iterates through every unique class representative in B' , it calculates the product of all quotient indices. Then, by [Lemma 3.11](#), multiplying the initial kernel order $\prod_{i=1}^s |E_i|!$ by these indices yields the order of the full automorphism group

$$N = \prod_{i=1}^s |E_i|! \cdot d_i = |G^{(E_s)}| \cdot \prod_{i=1}^s |G^{(E_{i-1})} : G^{(E_i)}| = |G|.$$

Algorithm 8: Reduction from ACOUNT to PCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$.

Output: The order of the automorphism group $|\text{Aut}(\mathcal{C})| \in \mathbb{N}$.

```
// Partition index set into classes of identical columns and construct set of
// unique column representatives.
Let  $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$  be the set of column vectors of  $\mathbf{G}$ .
 $\mathcal{E} := \emptyset$ . // collection of redundancy sets
 $\mathcal{G} := \emptyset$ . // keep track of unique columns
 $B' := \emptyset$ . // indices of first appearances

for  $i \leftarrow 1$  to  $n$  do
    // If a column matches a previously seen column, sort it into the same class.
    if  $\mathbf{g}_i \in \mathcal{G}$  then
        for  $E \in \mathcal{E}$  do
             $\mathbf{v} \leftarrow E[1]$ .
            if  $\mathbf{g}_i = \mathbf{v}$  then
                 $E \leftarrow E \cup \{i\}$ .
        // Otherwise, if a column is new, create a new class.
    else
         $E := \{i\}$ . // if
         $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ 
         $\mathcal{E} \leftarrow \mathcal{E} \cup \{E\}$ 
         $B' \leftarrow B' \cup \{i\}$ . // record index of first appearance

 $m \leftarrow 0$ . // maximum column multiplicity in  $\mathbf{G}$ 
 $N \leftarrow 1$ . // initialize the automorphism count
// compute the maximum column multiplicity
for  $E \in \mathcal{E}$  do
     $N \leftarrow N \cdot |E|!$ 
    if  $|E| > m$  then
         $m \leftarrow |E|$ 

// Find orbit partition of index set via APART oracle.
Run APART( $\mathbf{G}$ ) to obtain coordinate orbits  $\{X_1, \dots, X_\ell\}$ .

 $\mathcal{G} \leftarrow \emptyset$ . // reset to track fixed columns in stabilizer chain
 $\mathbf{G}_{\text{fixed}} \leftarrow \mathbf{G}$ . // initialize global base matrix
(continued on next page)
```

(Complexity) We first bound the number of oracle calls. The algorithm makes a single query to the APART oracle to obtain the orbit partition of the index set $[n]$. As shown in [Theorem 4.2](#), APART can be solved using $O(n^2)$ queries to a PCE oracle. In the main for-loop, a target $t_j \in B'$ is tested against a base index $t_i \in B'$ only if both belong to the same orbit X . For any orbit X , the number of distinct column classes is bounded by $|X|$. Thus, the outer for-loop evaluates the test branch at most $|X|$ times for this orbit, and each inner for-loop tests at most $|X| - 1$ targets. This limits the number of PCE oracle queries to $O(|X|^2)$ for each orbit X . Because the coordinate orbits form a partition of $[n]$, the sum of $O(|X|^2)$ across all orbits X is strictly bounded above by $O(n^2)$. Therefore, the algorithm makes a total of $O(n^2)$ calls to the PCE oracle.

Now we bound the runtime. Partitioning the index set $[n]$ into redundancy classes and constructing the set B' requires comparing column vectors at most n^2 times, so this step requires $O(n^2 \log(n))$ time. By [Proposition 4.3](#), running the APART oracle requires $O(n^3 \log(q))$ time. Now we evaluate the cost of constructing the test matrices, which dominates the time complexity of the algorithm. The maximum column multiplicity in $\mathbf{G}$ is $m \leq n$, and in each iteration $t_i \in B'$, the number of unique column classes fixed so far is $l \leq n$. The base matrix $\mathbf{G}_{\text{fixed}}$ of setwise stabilized columns accumulates $\sum_{a=1}^l a \cdot m$ appended columns in each iteration. To construct $\mathbf{G}_A$ and $\mathbf{G}_B$, an additional block of $(l+1)m$ columns is appended to $\mathbf{G}_{\text{fixed}}$. Thus, the number of columns appended to $\mathbf{G}$ is bounded above by $n + ml(l+1)/2 + (l+1)m = O(n^3)$.

(continued from previous page)

// Compute $|G^{(E_{i-1})} : G^{(E_i)}|$ for each consecutive pair of subgroups in the chain.

for $i \in B'$ do

$X :=$ unique orbit in $\{X_1, \dots, X_\ell\}$ containing i .

$d_i \leftarrow 1$. // initialize local orbit count

 // Check if an automorphism maps column i to non-identical column j .

 for $j \in X \cap B'$ where $j > i$ do

$l \leftarrow |\mathcal{G}|$. // number of unique classes fixed so far

$\mathbf{A} := (l+1)m$ copies of $\mathbf{g}_i$.

$\mathbf{B} := (l+1)m$ copies of $\mathbf{g}_j$.

$\mathbf{G}_A \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$.

$\mathbf{G}_B \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{B}]$.

 if $\text{PCE}(\mathbf{G}_A, \mathbf{G}_B) = \text{YES}$ then

$d_i \leftarrow d_i + 1$.

$N \leftarrow N \cdot d_i$. // update automorphism count

 // Stabilize set E_i for all future iterations.

$\mathbf{A} := (l+1)m$ copies of $\mathbf{g}_i$.

$\mathbf{G}_{\text{fixed}} \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$.

$\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$.

return N .

Each field element in $\mathbb{F}_q$ requires $\log(q)$ bits to represent and $O(\log(q))$ bit operations to read, copy, or compare. The initial partitioning phase and computing the kernel size requires $O(n^2 \log(q))$ and $O(n)$ operations respectively. Then, for each of the $O(n^2)$ oracle queries, copying at most $O(n^3)$ column vectors for the augmented matrices requires $O(kn^3 \log(q))$ operations. Therefore, since $k \leq n$, the total runtime is bounded by $O(n^2) \cdot O(kn^3 \log(q)) = O(n^6 \log(q))$ time. $\square$

5.4 MACOUNT to LCE

Now we prove the monomial analog of [Proposition 5.3](#). The reduction algorithm is nearly identical to [Algorithm 8](#) but uses the properties of monomial equivalence and a LCE oracle instead of PCE. For completeness, we include the full algorithm and proof below.

Proposition 5.4. *Algorithm 9 correctly solves MACOUNT for any q -ary n -length code in $O(n^6 \log(q))$ time using at most $O(n^2)$ calls to an LCE oracle.*

Proof [Algorithm 9](#) is nearly identical to [Algorithm 8](#) with the exception of an additional step to find the number c of indecomposable summands via [Algorithm 1](#). The proof of correctness is identical to that of [Proposition 5.3](#), but replacing [Lemmas 2.1](#) and [3.11](#) with their monomial analogs [Lemmas 2.2](#) and [3.13](#).

The time complexity remains $O(n^6 \log(q))$ because the $O(n^3 \log(q))$ runtime of Step 1 is dominated by the cost of constructing the augmented test matrices in Step 2. $\square$

Algorithm 9: Reduction from MACOUNT to LCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$.
Output: The order of the monomial automorphism group $|\text{MAut}(\mathcal{C})| \in \mathbb{N}$.

```
// Step 1: Code decomposition to find the number of independent summands.
Run Algorithm 1 on  $\mathbf{G}$  to obtain the decomposition  $\mathcal{C} \cong_{\mathcal{M}} \bigoplus_{r=1}^c A_r$ .
 $c :=$  number of indecomposable summands.

// Step 2: Partition index set into projective redundancy classes.
Let  $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$  be the set of column vectors of  $\mathbf{G}$ .
 $\mathcal{E} := \emptyset$ . // collection of redundancy sets
 $\mathcal{G} := \emptyset$ . // keep track of unique projective column representatives
 $B' := \emptyset$ . // indices of first appearances

for  $i \leftarrow 1$  to  $n$  do
    // If a column is a scalar multiple of a previously seen column, sort it into
    // the same class.
    if  $\exists \mathbf{v} \in \mathcal{G}$  such that  $\mathbf{g}_i = \lambda \mathbf{v}$  for some  $\lambda \in \mathbb{F}_q^*$  then
        for  $E \in \mathcal{E}$  do
             $\mathbf{v} \leftarrow E[1]$ .
            if  $\mathbf{g}_i = \lambda \mathbf{v}$  for some  $\lambda \in \mathbb{F}_q^*$  then
                 $E \leftarrow E \cup \{\mathbf{g}_i\}$ .
        // Otherwise, if a column is unique up to scaling, create a new class.
    else
         $E := \{\mathbf{g}_i\}$ .
         $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ 
         $\mathcal{E} \leftarrow \mathcal{E} \cup \{E\}$ 
         $B' \leftarrow B' \cup \{i\}$ . // record index of first appearance

 $m \leftarrow 0$ . // maximum projective column multiplicity in  $\mathbf{G}$ 
 $N \leftarrow (q-1)^c$ . // initialize count with the global scaling automorphisms
// count the permutations stabilizing the projective classes
for  $E \in \mathcal{E}$  do
     $N \leftarrow N \cdot |E|!$ 
    if  $|E| > m$  then
         $m \leftarrow |E|$ 

// Step 3: Find the monomial orbit partition of the index set via MAPART oracle.
Run MAPART( $\mathbf{G}$ ) to obtain monomial coordinate orbits  $\{X_1, \dots, X_\ell\}$ .
 $\mathcal{G} \leftarrow \emptyset$ . // reset to track fixed columns in stabilizer chain
 $\mathbf{G}_{\text{fixed}} \leftarrow \mathbf{G}$ . // initialize global base matrix
(continued on next page)
```

(continued from previous page)

```

// Step 4: Compute index  $|G^{(E_{i-1})} : G^{(E_i)}|$  for each consecutive pair of subgroups
// in the chain.
for  $i \in B'$  do
     $X :=$  unique orbit in  $\{X_1, \dots, X_\ell\}$  containing  $i$ .
     $d_i \leftarrow 1$ . // initialize local orbit count
    // Check if an automorphism maps column  $i$  to a valid spatial target  $j$ .
    for  $j \in X \cap B'$  where  $j > i$  do
         $l \leftarrow |\mathcal{G}|$ . // number of unique projective classes fixed so far
         $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
         $\mathbf{B} := (l+1)m$  copies of  $\mathbf{g}_j$ .
         $\mathbf{G}_A \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ .
         $\mathbf{G}_B \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{B}]$ .
        // Use the LCE oracle to check for scalar transformations that fix
        // projective class of  $\mathbf{g}_i$ .
        if LCE( $\mathbf{G}_A, \mathbf{G}_B$ ) = YES then
             $d_i \leftarrow d_i + 1$ .
     $N \leftarrow N \cdot d_i$ . // update total monomial automorphism count
    // Stabilize projective class  $E_i$  for all future iterations.
     $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
     $\mathbf{G}_{\text{fixed}} \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ .
     $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ .

return  $N$ .
```

5.5 ICOUNT and MICOUNT

Here we present the isomorphism analog of the reductions between code equivalence and the automorphism counting problem. ICOUNT (resp. MICOUNT) is the problem of counting the number of (resp. monomial) isomorphisms that exist between two input codes (see [Definition 2.7](#)).

Theorem 5.3 (PCE Reduces to ICOUNT, LCE Reduces to MICOUNT). *There is a reduction from PCE (resp. LCE) on any pair of n -length q -ary codes to ICOUNT (resp. MICOUNT) that runs in $O(n^2 \log(q))$ time and makes a single oracle call.*

Proof Given generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$, make one oracle call ICOUNT($\mathcal{C}_1, \mathcal{C}_2$). If the output is 0, then the given codes are not equivalent, and if it outputs a positive value, then the given codes are equivalent. An identical reduction can be made in which a MICOUNT oracle is called once to solve LCE. Copying the two $k \times n$ generator matrices over $\mathbb{F}_q$ requires $2nk \log(q) = O(n^2 \log(q))$ time. $\square$

For the reverse reductions from ICOUNT to PCE (and MICOUNT to LCE), we use a similar approach as that in [\[10\]](#) and rely on the relationship between (monomial) isomorphisms and automorphisms described in [Lemmas 2.4](#) and [2.5](#).

Corollary 5.1 (ICOUNT Reduces to PCE, MICOUNT Reduces to LCE). *There is a reduction from ICOUNT (resp. MICOUNT) on any pair of n -length q -ary codes to PCE (resp. LCE) that runs in $O(n^6 \log(q))$ time and makes $O(n^2)$ oracle calls.*

Proof Let $\mathbf{G}_1$ and $\mathbf{G}_2$ be the input generator matrices for codes $\mathcal{C}_1$ and $\mathcal{C}_2$. The reduction algorithm first makes a single call to the PCE oracle to check whether $\mathcal{C}_1 \cong \mathcal{C}_2$. If PCE($\mathbf{G}_1, \mathbf{G}_2$) outputs NO, then no isomorphism exists between the codes and $\text{Iso}(\mathcal{C}_1, \mathcal{C}_2) = 0$. In this case, the reduction algorithm simply outputs 0. If PCE($\mathbf{G}_1, \mathbf{G}_2$) outputs YES, then there exists at least one isomorphism from $\mathcal{C}_1$ to $\mathcal{C}_2$. By [Lemma 2.4](#), we have $|\text{Iso}(\mathcal{C}_1, \mathcal{C}_2)| = |\text{Aut}(\mathcal{C}_2)|$. To find this cardinality, we run [Algorithm 8](#) on $\mathbf{G}_2$ to obtain $|\text{Aut}(\mathcal{C}_2)|$.

Reading and copying the two $k \times n$ matrices over $\mathbb{F}_q$ requires $2nk \log(q) = O(n^2 \log(q))$ time. Checking equivalence requires 1 oracle call to PCE. In the event that the PCE oracle outputs YES, the algorithm runs [Algorithm 8](#), which requires $O(n^2)$ additional oracle calls to PCE and runs in $O(n^3 \log(q))$ time. Thus, in the worst case, the reduction algorithm requires $1 + O(n^2) = O(n^2)$ oracle calls to PCE and $O(n^3 \log(q))$ time.

A nearly identical reduction can be made for MICOUNT using an LCE oracle. The LCE oracle is called once to check whether $C_1 \cong_{\mathcal{M}} C_2$. If $\text{LCE}(\mathbf{G}_1, \mathbf{G}_2)$ outputs NO, then we output 0. Otherwise, if $\text{LCE}(\mathbf{G}_1, \mathbf{G}_2)$ outputs YES, [Lemma 2.5](#) guarantees that $|\text{Isom}(C_1, C_2)| = |\text{MAut}(C_2)|$. We run [Algorithm 9](#) on $\mathbf{G}_2$ to compute this cardinality. As with the permutation case, this requires $1 + O(n^2) = O(n^2)$ oracle calls to LCE and $O(n^6 \log(q))$ time, the bottleneck being the runtime of [Algorithm 9](#). $\square$

6 AGEN and MAGEN

Theorem 6.1 (search-PCE Reduces to AGEN, search-LCE Reduces to MAGEN). *There is a reduction from search-PCE (resp. search-LCE) on any pair of n -length q -ary codes C_1, C_2 to AGEN (resp. MAGEN) on their direct sum $\mathcal{C} := C_1 \oplus C_2$ that makes a single oracle call and runs in $O(sn^2 \log(q) + n^4)$ time, where s is the size of the generating set found by the oracle.*

Theorem 6.2 (AGEN Reduces to PCE, MAGEN Reduces to LCE). *There is a reduction from AGEN (resp. MAGEN) on any n -length q -ary code that runs in $O(n^6 \log(q))$ time and makes $O(n^4)$ oracle calls to PCE (resp. LCE).*

6.1 Search-PCE Reduces to AGEN

In this section, we prove that finding an isomorphism between two codes, if it exists, is a polynomially equivalent problem to that of finding a generating set for the automorphism group of their direct sum code.

At a high-level, the idea for the reduction is similar to that in [Algorithm 6](#) for reducing PCE to ACOUNT. This reduction algorithm also decomposes the two input codes C_1, C_2 into direct sums of indecomposable codes and searches for pairs of isomorphic summands. In this case, the algorithm finds a generating set for the direct sum code $C_1 \oplus C_2$ using the AGEN oracle. Every automorphism for $C_1 \oplus C_2$ corresponds to an isomorphism between a summand of C_1 and a summand of C_2 . If there is a bijection between the summands of C_1 and the summands of C_2 , then the algorithm constructs an isomorphism between C_1 and C_2 .

Proposition 6.1. *[Algorithm 10](#) correctly solves search-PCE for any pair of n -length codes in $O(sn^2 \log(q) + n^4)$ time using a single call to an AGEN oracle, where s is the size of the generating set found by the oracle.*

Proof Let $C_1, C_2 \subseteq \mathbb{F}_q^n$ be the two input codes.

(Correctness) To prove that [Algorithm 10](#) is correct, we must show that if $C_1 \cong C_2$, the algorithm outputs an isomorphism π between the two codes, and outputs $\perp$ otherwise.

In Step 1, [Algorithm 1](#) is applied to the input generator matrices to decompose the codes into direct sums of indecomposable codes. This yields $\pi_1(C_1) = A_1 \oplus \dots \oplus A_{m_1}$ and $\pi_2(C_2) = B_1 \oplus \dots \oplus B_{m_2}$ for some $m_1, m_2 \leq n$ and indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$. By [Lemma 3.1](#), the codes can only be equivalent if they have the same number of summands and $m_1 = m_2$. If this is the case, we denote the corresponding index sets by $\{X_i\}_{i=1}^m$ and $\{Y_j\}_{j=1}^m$.

In Step 2, the algorithm constructs the direct sum code $\mathcal{C} = \pi_1(C_1) \oplus \pi_2(C_2)$. Let $Z_1 := [n]$ and $Z_2 := \{n+1, \dots, 2n\}$ denote the index sets of $\pi_1(C_1)$ and $\pi_2(C_2)$ in $\mathcal{C}$, respectively. These form a partition $[2n] = Z_1 \sqcup Z_2$. The AGEN oracle is then called once on $\mathcal{C}$ to obtain a set S of automorphisms that generate the group $\text{Aut}(\mathcal{C})$. By definition, any automorphism of $\mathcal{C}$ can be written as a composition of generators in S .

In Step 3, we construct a directed graph G' that represents all equivalences between the indecomposable summands indicated by the generating set S . The summands $\{A_i\}_{i=1}^m$ and $\{B_j\}_{j=1}^m$ are represented by the vertices $\{a_1, \dots, a_m\}$ and $\{b_1, \dots, b_m\}$, respectively. The algorithm then iterates through all s generators in S . For each generator $\phi \in S$, it finds the image of each summand A_i under ϕ , given by the restriction $\phi|_{X_i}(A_i)$. By [Corollary 3.1](#), if $\{A_i\}_{i=1}^m$ and $\{B_j\}_{j=1}^m$ are decompositions of equivalent codes, then every A_i must map either to some A_j or B_j under ϕ . A directed edge is added to G' between the vertex for A_i and its image: If $\phi|_{X_i}(A_i) = B_j$ for some $j \in [m]$, then the edge (a_i, b_j) is added. Otherwise if $\phi|_{X_i}(A_i) = A_j$, edge (a_i, a_j) is

added; in the event that A_i maps to itself, no self-loop edge is added so that the final graph G' is a simple graph. The algorithm keeps track of the automorphisms in S that maps each summand to another, using a dictionary object *maps* which functions as a set of labels for each edge. While there may be multiple automorphisms that map A_i to B_j (or A_j for $j \neq i$), the algorithm updates the corresponding dictionary entry to keep only the most recently encountered one. By construction, the final graph G' contains edges only between equivalent summands and does not include self-loops or multiple edges between vertices.

In Step 4, the algorithm constructs a bipartite graph G using the equivalence graph G' . For each summand A_i , it runs Dijkstra's algorithm to find a shortest path between the vertex a_i and every other node in its connected component, G_{a_i} . For every vertex b_j in V that is reachable from a_i , the algorithm adds an edge (a_i, b_j) to the

Algorithm 10: Reduction from search-PCE to AGEN

Input : Generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: An isomorphism $\sigma \in \mathcal{S}_n$ such that $\sigma(\mathcal{C}_1) = \mathcal{C}_2$.

// Step 1: Code Decomposition

Run [Algorithm 1](#) on $\mathbf{G}_1, \mathbf{G}_2$ to obtain decompositions $\pi_1(\mathcal{C}_1) = \bigoplus_{i=1}^{m_1} A_i$ and $\pi_2(\mathcal{C}_2) = \bigoplus_{j=1}^{m_2} B_j$,
for some indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$ and permutations $\pi_1, \pi_2 \in \mathcal{S}_n$.

if $m_1 \neq m_2$ **then**

return $\perp$.

Let $m := m_1 = m_2$.

Denote the index sets of $\mathcal{C}_1$ and $\mathcal{C}_2$ by $\{X_i\}_{i=1}^m$ and $\{Y_j\}_{j=1}^m$.

// Step 2: AGEN Oracle Call

Construct the direct sum code $\mathcal{C} := \pi_1(\mathcal{C}_1) \oplus \pi_2(\mathcal{C}_2)$.

Run AGEN on $\mathcal{C}$ to obtain a generating set $S = \{\phi_1, \dots, \phi_s\}$ of $\text{Aut}(\mathcal{C})$.

// Step 3: Construction of the Equivalence Graph

Define the vertex sets $U := \{a_1, \dots, a_m\}$ and $V := \{b_1, \dots, b_m\}$.

Set $E' := \emptyset$.

Set *maps* := $\{\}$. *// dictionary of equivalence maps*

for $\phi \in S$ **do**

for $i \leftarrow 1$ **to** m **do**

$\text{Im}(A_i) := \phi|_{X_i}(A_i)$.

$\text{Im}(B_i) := \phi|_{Y_i}(B_i)$.

// Find where a_i maps to.

for $j \leftarrow 1$ **to** m **do**

if $\text{Im}(A_i) = B_j$ **then**

$E' \leftarrow E' \cup \{(a_i, b_j)\}$. *// add directed edge*

$\text{maps}[(a_i, b_j)] := \phi$. *// update edge label*

else if $\text{Im}(A_i) = A_j$ **and** $j \neq i$ **then**

$E' \leftarrow E' \cup \{(a_i, a_j)\}$. *// add directed edge*

$\text{maps}[(a_i, a_j)] := \phi$. *// update edge label*

// Find where b_i maps to.

for $j \leftarrow 1$ **to** m **do**

if $\text{Im}(B_i) = A_j$ **then**

$E' \leftarrow E' \cup \{(b_i, a_j)\}$. *// add directed edge*

$\text{maps}[(b_i, a_j)] := \phi$. *// update edge label*

else if $\text{Im}(B_i) = B_j$ **and** $j \neq i$ **then**

$E' \leftarrow E' \cup \{(b_i, b_j)\}$. *// add directed edge*

$\text{maps}[(b_i, b_j)] := \phi$. *// update edge label*

Define graph $G' := (U \cup V, E')$.

(continued on next page)

(continued from previous page)

```

// Step 4: Construction of the Bipartite Graph
Set  $E := \emptyset$ .
Set  $\text{equivmaps} := \{\}$ . // edge labels
Let  $G_{a_i}$  denote the connected component in  $G'$  that contains  $a_i$ .
Run DIJKSTRA'S ALGORITHM on  $G'$  from root  $a_i$  to obtain paths  $\{P_v \subseteq E' : v \in G_{a_i}\}$  to all
vertices in its connected component  $G_{a_i}$ .

for  $v \in G_{a_i} \cap V$  do
     $j \leftarrow V.\text{index}(v)$ . //  $v = b_j$  for some  $j \in [m]$ 
     $\psi_{i,j} \leftarrow \text{id}$ . // global automorphism
    for  $e \in P_v$  do
         $\phi_e \leftarrow \text{maps}[e]$ . // get directed edge label
         $\psi_{i,j} \leftarrow \phi_e \circ \psi_{i,j}$ .
     $E \leftarrow E \cup \{(a_i, b_j)\}$ .
     $\psi_{i,j} \leftarrow \psi_{i,j}|_{X_i}$ . // restrict to support of  $A_i$ 
     $\text{equivmaps}[(a_i, b_j)] \leftarrow \psi_{i,j}$ . // update edge label
Define graph  $G := (U \sqcup V, E)$ .

// Step 5: Perfect Matching between Summands
Run the HOPCROFT-KARP ALGORITHM on  $G$  to obtain a maximum bipartite matching  $M \subseteq E$ .
if  $|M| \neq m$  then
    return  $\perp$ .

// Find the isomorphism corresponding to the perfect matching.
Let  $\sigma : [n] \rightarrow [n]$ .
// Define isomorphism from  $\mathcal{C}_1$  to  $\mathcal{C}_2$  piecewise
for  $(i, j) \in M$  do
     $\psi_{i,j} \leftarrow \text{equivmaps}[(a_i, b_j)]$ . // get edge label
     $\sigma|_{X_i} := \psi_{i,j}$ . // image of  $i$ -th set of the partition of  $[n]$ 
return  $\pi = \pi_2^{-1} \sigma \pi_1$ . // compose isomorphism with equivalence maps

```

new graph G . Then, it traverses the path P_{b_j} from a_i to b_j , retrieves the automorphism label for each edge in this path, and composes the sequence of automorphisms to obtain an automorphism that maps A_i to B_j . For example, if $\{e_1, \dots, e_r\}$ is a path from a_i to b_j , the corresponding sequence of generators $\phi_{e_1}, \dots, \phi_{e_r}$ is composed into $\psi_{i,j} := (\phi_{e_r} \circ \dots \circ \phi_{e_2} \circ \phi_{e_1})|_{X_i}$. By [Corollary 3.1](#), the images of the summands are disjoint, so this composition is a well-defined bijection that satisfies $\psi_{i,j}(A_i) = B_j$. By construction, the resulting graph G is bipartite on vertex sets U and V , and contains one edge for every equivalent pair of summands belonging to different codes.

Suppose that $\mathcal{C}_1 \cong \mathcal{C}_2$. In this case, we have $\mathcal{C}_1 \cong A_1 \oplus \dots \oplus A_m \cong B_1 \oplus \dots \oplus B_m \cong \mathcal{C}_2$. By [Lemma 3.1](#), there exists an isomorphism map $\pi : \pi_1(\mathcal{C}_1) \rightarrow \pi_2(\mathcal{C}_2)$ that reorders the summands $\{A_i\}_{i=1}^m$ and maps each one to an equivalent summand in $\{B_j\}_{j=1}^m$, i.e. $\pi|_{X_i}(A_i) = B_{\sigma_\pi(i)}$ for all $i \in [m]$ for some permutation $\sigma_\pi : [m] \rightarrow [m]$ induced by π . By construction of $\mathcal{C}$, the group $\text{Aut}(\mathcal{C})$ must contain some $\pi' \in \text{Aut}(\mathcal{C})$ such that $\pi'|_{Z_1}(\pi_1(\mathcal{C}_1)) = \pi_2(\mathcal{C}_2) = \pi(\pi_1(\mathcal{C}_1))$ (and symmetrically, $\pi'|_{Z_2}(\pi_2(\mathcal{C}_2)) = \pi_1(\mathcal{C}_1) = \pi^{-1}(\pi_2(\mathcal{C}_2))$). We show that the algorithm outputs π .

In Step 5, the algorithm explores the bipartite graph G and runs the Hopcroft-Karp algorithm from [\[42\]](#) to find a maximum bipartite matching M between the vertices $a_1, \dots, a_m$ and $b_1, \dots, b_m$. If the matching is not perfect and has fewer than m edges, then no isomorphism exists from $\{A_i\}_{i=1}^m$ to $\{B_j\}_{j=1}^m$. If $|M| = m$, then we have a bijective map between $\{A_i\}_{i=1}^m$ and $\{B_j\}_{j=1}^m$. The automorphism labels for the edges in the matching M are then pieced together to obtain a isomorphism π from $\pi_1(\mathcal{C}_1)$ to $\pi_2(\mathcal{C}_2)$. By construction, the automorphisms for the edges in G , and hence M , have disjoint supports, so their composition is a well-defined bijection that maps $\pi(\pi_1(\mathcal{C}_1)) = \pi_2(\mathcal{C}_2)$. Finally, composing this isomorphism with the equivalence maps yields an isomorphism $\sigma = \pi_2^{-1} \pi \pi_1$ that satisfies

$$\sigma(\mathcal{C}_1) = \pi_2^{-1} \pi \pi_1(\mathcal{C}_1) = \pi_2^{-1}(\pi_2(\mathcal{C}_2)) = \mathcal{C}_2.$$

(Complexity) We analyze the runtime for each step. Step 1 runs [Algorithm 1](#) on both input codes, which requires $2 \cdot O(n^3 \log(q)) = O(n^3 \log(q))$ time. In Step 2, the direct sum code $\mathcal{C} = \mathcal{C}_1 \oplus \mathcal{C}_2$ can be constructed from the given generator matrices in $O(n \log(q))$ time. A single oracle call to **AGEN** on $\mathcal{C}$ yields a generating set

S for its automorphism group of size $s := |S|$. The equivalence graph G' constructed in Step 3 requires iterating over all s generators in S and every pair of summands in the sets $\{A_i\}_{i=1}^m$ and $\{B_j\}_{j=1}^m$. This requires a total of $O(s \cdot m^2 \log(q))$ time. In Step 4, the algorithm runs Dijkstra's algorithm starting from a_i for each of the m vertices in U . In the worst case, when all the summands are equivalent to each other, G' will be a connected graph; this requires every vertex to be explored in each iteration. Hence, finding shortest paths from a_i to every other vertex in its connected component G_{a_i} takes at most $\Theta(|E'| + |U \cup V|^2) = \Theta(m(2m-1) + (2m)^2) = O(m^2)$ time. After exploring the connected component G_{a_i} , the algorithm iterates through each vertex b_j in $G_{a_i} \cap V$, of which there are at most m . Then it traverses the path between a_i and all such b_j , which contains at most $m(2m-1)$ edges. So, for all m vertices in U , this step requires at most $m \cdot (O(m^2) + O(m \cdot m(2m-1))) = O(m^4)$ time. Finally, in Step 5, running the Hopcroft-Karp algorithm on the graph G with $|V| = 2m \leq 2n$ vertices and $|E| \leq 2m^2 \leq 2n^2$ edges requires at most $O(|E| \cdot \sqrt{|V|}) = O(m^{2.5}) = O(n^{2.5})$ time. In the case where this outputs a perfect matching M of size m , iterating through each of the m edges and composing the automorphisms takes $O(m)$ time.

Therefore, the total asymptotic runtime of Algorithm 10 is $O(n^3 \log(q)) + O(n \log(q)) + O(s \cdot m^2 \log(q)) + O(m^4) + O(m^{2.5}) = O(s \cdot n^2 \log(q) + n^4)$. $\square$

6.2 Search-LCE Reduces to MAGEN

Now we prove the monomial analog of Proposition 6.1. The reduction algorithm is nearly identical to Algorithm 10 but uses the properties of monomial equivalence and a MAGEN oracle instead of AGEN. We include the algorithm and its proof here for completeness.

Proposition 6.2. *Algorithm 11 correctly solves search-LCE for any pair of n -length codes in $O(s \cdot n^2 \log(q) + n^4)$ time using a single call to an MAGEN oracle, where s is the size of the generating set found by the oracle.*

Proof The proof is structurally identical to that of Proposition 6.1. By running the MAGEN oracle on the direct sum code $\mathcal{C} := \pi_1(\mathcal{C}_1) \oplus \pi_2(\mathcal{C}_2)$, we obtain a generating set for $\text{MAut}(\mathcal{C})$ that contains the linear isometries mapping the summands of $\mathcal{C}_1$ to $\mathcal{C}_2$. Because the monomial automorphism group action preserves the direct sum decomposition, as in Algorithm 10, the equivalence graph construction and shortest-path search produce a mapping between the indecomposable summands (if it exists). Piecewise composition of the maps corresponding to the edges of the shortest path in the graph yields the final linear isometry between $\mathcal{C}_1$ to $\mathcal{C}_2$.

A single oracle call is made to the MAGEN oracle. For the time complexity, finding the shortest paths and extracting the maximum bipartite matching requires the same time as for Algorithm 10; this is dominated by the $O(s \cdot n^2 \log(q) + n^4)$ time required to process the s monomial generators. $\square$

Algorithm 11: Reduction from search-LCE to MAGEN

Input : Generator matrices $\mathbf{G}_1, \mathbf{G}_2 \in \mathbb{F}_q^{k \times n}$ for codes $\mathcal{C}_1, \mathcal{C}_2 \subseteq \mathbb{F}_q^n$.

Output: A linear isometry $\pi \in (\mathbb{F}_q^*)^n \rtimes \mathcal{S}_n$ such that $\pi(\mathcal{C}_1) = \mathcal{C}_2$.

// Step 1: Code Decomposition

Run Algorithm 1 on $\mathbf{G}_1, \mathbf{G}_2$ to obtain decompositions $\pi_1(\mathcal{C}_1) = \bigoplus_{i=1}^{m_1} A_i$ and $\pi_2(\mathcal{C}_2) = \bigoplus_{j=1}^{m_2} B_j$,
for some indecomposable codes $\{A_i\}_{i=1}^{m_1}$ and $\{B_j\}_{j=1}^{m_2}$ and permutations $\pi_1, \pi_2 \in \mathcal{S}_n$.

if $m_1 \neq m_2$ then

 return $\perp$.

Let $m := m_1 = m_2$.

Denote the index sets of $\mathcal{C}_1$ and $\mathcal{C}_2$ by $\{X_i\}_{i=1}^m$ and $\{Y_j\}_{j=1}^m$.

// Step 2: MAGEN Oracle Call

Construct the direct sum code $\mathcal{C} := \pi_1(\mathcal{C}_1) \oplus \pi_2(\mathcal{C}_2)$.

Run MAGEN on $\mathcal{C}$ to obtain a generating set $S = \{\phi_1, \dots, \phi_s\}$ of $\text{MAut}(\mathcal{C})$.

// Step 3: Construction of the Equivalence Graph

Define the vertex sets $U := \{a_1, \dots, a_m\}$ and $V := \{b_1, \dots, b_m\}$.

Set $E' := \emptyset$.

Set $\text{maps} := \{\}$. // dictionary of equivalence maps

(continued on next page)

(continued from previous page)

```

for  $\phi \in S$  do
  for  $i \leftarrow 1$  to  $m$  do
     $Im(A_i) := \phi|_{X_i}(A_i)$ .
     $Im(B_i) := \phi|_{Y_i}(B_i)$ .
    // Find where  $a_i$  maps to.
    for  $j \leftarrow 1$  to  $m$  do
      if  $Im(A_i) = B_j$  then
         $E' \leftarrow E' \cup \{(a_i, b_j)\}$ .
         $maps[(a_i, b_j)] := \phi$ . // update edge label
      else if  $Im(A_i) = A_j$  and  $j \neq i$  then
         $E' \leftarrow E' \cup \{(a_i, a_j)\}$ .
         $maps[(a_i, a_j)] := \phi$ . // update edge label
    // Find where  $b_i$  maps to.
    for  $j \leftarrow 1$  to  $m$  do
      if  $Im(B_i) = A_j$  then
         $E' \leftarrow E' \cup \{(b_i, a_j)\}$ .
         $maps[(b_i, a_j)] := \phi$ . // update edge label
      else if  $Im(B_i) = B_j$  and  $j \neq i$  then
         $E' \leftarrow E' \cup \{(b_i, b_j)\}$ .
         $maps[(b_i, b_j)] := \phi$ . // update edge label

```

Define graph $G' := (U \sqcup V, E')$.

// Step 4: Construction of the Bipartite Graph

Set $E := \emptyset$.

Set $equivmaps := \{\}$. // edge labels

for $i \leftarrow 1$ **to** m **do**

Let G_{a_i} denote the connected component in G' that contains a_i .

Run DIJKSTRA'S ALGORITHM on G' from root a_i to obtain paths $\{P_v \subseteq E' : v \in G_{a_i}\}$ to all vertices in its connected component G_{a_i} .

for $v \in G_{a_i} \cap V$ **do**

$j := V.index(v)$. // $v = b_j$ for some index $j \in [m]$.

$\psi_{i,j} := (1, id)$. // global monomial automorphism

for $e \in P_v$ **do**

$\phi_e := maps[e]$. // get edge label

$\psi_{i,j} \leftarrow \phi_e \circ \psi_{i,j}$. // compose monomial automorphisms

$E \leftarrow E \cup \{(a_i, b_j)\}$.

$\psi_{i,j} \leftarrow \psi_{i,j}|_{X_i}$. // restrict to support of A_i

$equivmaps[(a_i, b_j)] := \psi_{i,j}$. // update edge label

Define graph $G := (U \sqcup V, E)$.

// Step 5: Perfect Matching between Summands

Run the HOPCROFT-KARP ALGORITHM on G to obtain a maximum bipartite matching $M \subseteq E$.

if $|M| \neq m$ **then**

return $\perp$.

// Find the linear isometry corresponding to the perfect matching.

Let $\sigma := (1, id)$. // initialize map to be defined

// Define the linear isometry from $\mathcal{C}_1$ to $\mathcal{C}_2$ piecewise

for $(i, j) \in M$ **do**

$\psi_{i,j} \leftarrow equivmaps[(a_i, b_j)]$. // get edge label

$\sigma|_{X_i} := \psi_{i,j}$. // image of i -th set of the partition of $[n]$

$\pi := (1, \pi_2^{-1}) \circ \sigma \circ (1, \pi_1)$. // compose linear isometry with permutation isomorphisms

return π .

6.3 AGEN Reduces to PCE

We now prove the reverse reduction, from AGEN to PCE. Note that AGEN reduces to search-PCE as a consequence, since there is a trivial reduction from PCE to search-PCE. The reduction algorithm relies on two oracles, for solving APART and search-PCE; these oracles can be replaced with [Algorithm 4](#) and the algorithm from [Lemma 1.1](#), which rely on a PCE oracle.

For any input code $\mathcal{C}$ with generator matrix $\mathbf{G}$, let $G := \text{Aut}(\mathcal{C})$ denote its permutation automorphism group and let $G = G^{(E_0)} \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ be the setwise stabilizer chain given by $\mathbf{G}$. The algorithm finds a generating set for G by finding a generating set for the smallest subgroup $G^{(E_s)}$ in the chain, together with a set of left coset representatives for the quotient space given by each pair of consecutive subgroups along the chain; this is given by [Lemma 3.14](#). The subgroup $G^{(E_s)}$ stabilizes every projective class $E_1, \dots, E_s$ setwise, and so contains all the automorphisms that swap indices belonging to the same class. This group is generated by all possible transpositions for every projective class. To find left coset representatives for $G^{(E_{i-1})}/G^{(E_i)}$ for every $1 \leq i \leq s$, the algorithm searches for automorphisms that maps indices from E_i to some E_j for every j in the orbit of i , under the condition that indices in $E_1, \dots, E_{i-1}$ are stabilized within their respective classes. To test whether such an automorphism exists, the algorithm invokes the PCE on two carefully-constructed test matrices which guarantee that any automorphism between them belongs to $G^{(E_{i-1})}/G^{(E_i)}$ and maps i to j .

Proposition 6.3. *Algorithm 12 correctly solves AGEN for any n -length q -ary code in $O(n^6 \log(q))$ time using 1 call to an APART oracle, $O(n^2)$ queries to a search-PCE oracle, and $O(n^2)$ additional calls to a PCE oracle.*

As an immediate consequence of [Theorem 4.2](#) and [Lemma 1.1](#), we obtain a reduction from AGEN to PCE that makes $O(n^4)$ oracle calls, as claimed in [Theorem 6.2](#).

Algorithm 12: Reduction from AGEN to PCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$.
Output: A generating set $S \subseteq \mathcal{S}_n$ for the automorphism group $\text{Aut}(\mathcal{C})$.

```
// Step 1: Partition index set into column classes and construct coord set  $B'$ .
Let  $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$  be the set of column vectors of  $\mathbf{G}$ .
 $\mathcal{E} := \emptyset$ . // collection of redundancy sets
 $seen := \emptyset$ . // keep track of unique columns
 $B' := \emptyset$ . // indices of first appearances
for  $i \leftarrow 1$  to  $n$  do
    if  $\mathbf{g}_i \in seen$  then
        for  $E \in \mathcal{E}$  do
             $\mathbf{v} \leftarrow E[1]$ .
            if  $\mathbf{g}_i = \mathbf{v}$  then
                 $E \leftarrow E \cup \{i\}$ .
    else
         $E := \{\mathbf{g}_i\}$ .
         $seen \leftarrow seen \cup \{\mathbf{g}_i\}$ .
         $\mathcal{E} \leftarrow \mathcal{E} \cup \{E\}$ .
         $B' \leftarrow B' \cup \{i\}$ . // record coord of new unique column
 $m \leftarrow 0$ .
 $S \leftarrow \emptyset$ . // initialize generating set
(continued on next page)
```

(continued from previous page)

```

// Step 2: Generate the smallest subgroup  $G^{(E_s)}$  using transpositions.
for  $E \in \mathcal{E}$  do
     $\mathbf{e}_1 \leftarrow E[1]$ .
    for  $\mathbf{e}_k \in E$  where  $\mathbf{e}_k \neq \mathbf{e}_1$  do
         $S \leftarrow S \cup \{(\mathbf{e}_1, \mathbf{e}_k)\}$ .
    // Find maximum column multiplicity.
    if  $|E| > m$  then
         $m \leftarrow |E|$ .

// Step 3: Find orbit partition using the APART oracle.
Run APART( $\mathbf{G}$ ) to obtain coordinate orbits  $X_1, \dots, X_\ell$ .

 $\mathcal{G} \leftarrow \emptyset$ . // track permanently fixed columns in subchain
 $\mathbf{G}_{\text{fixed}} \leftarrow \mathbf{G}$ . // initialize global base matrix
// Step 4: Compute all coset transversals  $\sigma \in R_i$  along setwise stabilizer chain.
for  $i \in B'$  do
     $X := \text{unique orbit in } \{X_1, \dots, X_\ell\} \text{ containing } i$ .
    // Find a mapping from  $E_i$  to each valid target class.
    for  $j \in X \cap B'$  where  $j > i$  do
         $l \leftarrow |\mathcal{G}|$ .
         $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
         $\mathbf{B} := (l+1)m$  copies of  $\mathbf{g}_j$ .
         $\mathbf{G}_A \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ .
         $\mathbf{G}_B \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{B}]$ .

        // Check if there exists an automorphism that maps  $i$  to  $j$ .
        if PCE( $\mathbf{G}_A, \mathbf{G}_B$ )  $\rightarrow$  YES then
            // If it exists, find it via the Search-PCE oracle.
            Run Search-PCE( $\mathbf{G}_A, \mathbf{G}_B$ ) to obtain a map  $\pi$ .
             $U \leftarrow \{1, \dots, n\}$ . // keep track of mapped indices
             $\sigma := \text{id}$ . // initialize generator
            for  $c \leftarrow 1$  to  $n$  do
                if  $\pi(c) \leq n$  then
                     $t := \pi(c)$ 
                else
                     $t := \text{coord of the first column in } \mathbf{G}_B \text{ identical to column } \pi(c)$ .
                 $t' := \min\{u \in U \mid \mathbf{g}_u = \mathbf{g}_t\}$ 
                 $\sigma(c) \leftarrow t'$ 
                 $U \leftarrow U \setminus \{t'\}$ 
             $S \leftarrow S \cup \{\sigma\}$ . // add transversal representative

         $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
         $\mathbf{G}_{\text{fixed}} \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ . // fix base point for next iteration
         $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ .

return  $S$ .
```

Proof Let $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ be the input generator matrix of the code $\mathcal{C}$, and $G = \text{Aut}(\mathcal{C})$ denote its automorphism group. Let $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$ denote the columns of $\mathbf{G}$ in order of appearance. We prove that [Algorithm 8](#) correctly outputs a generating set S for G by traversing a setwise stabilizer chain for G and applying [Lemma 3.14](#).

(Correctness) The algorithm first partitions the columns of $\mathbf{G}$ into disjoint redundancy classes $\mathcal{E} = \{E_1, \dots, E_s\}$, where each E_i contains identical columns of a particular type, and no two classes contain the same vector. For this sorting process, the algorithm identifies the coordinates of the unique column vectors in $\mathbf{G}$ in

order of first appearance; let $B' = \{t_1, \dots, t_s\} \subseteq [n]$ denote the elements of this subset. Note that each coordinate $t_i \in B'$ defines the redundancy class $E_i \in \mathcal{E}$. By Definition 3.2, this B' defines a setwise stabilizer chain $G \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$. By definition, the smallest subgroup $G^{(E_s)}$ contains all permutations that trivially swaps columns in each redundancy class. Then, every automorphism in $G^{(E_s)}$ can be expressed as a composition of transpositions, so a generating set for $G^{(E_s)}$ consists of all 2-cycle permutations for each redundancy class. The algorithm finds and adds all these transpositions to the final generating set S , and so it includes a generating subset S_{E_s} for $G^{(E_s)}$. The maximum column multiplicity m in $\mathbf{G}$ is then computed by finding the cardinality of the largest redundancy class.

Next, the algorithm makes a single oracle call to an APART oracle to obtain a partition of the index set $[n]$ into orbits under automorphisms in G . While not strictly necessary, this step provides an optimization that restricts the search space for automorphisms in the following step.

To find a generating set for the remaining automorphisms in G , the algorithm traverses the chain $G \geq G^{(E_1)} \geq \dots \geq G^{(E_s)}$ in descending order by iterating through the elements of B' . In the t_i -th iteration, we find the coset representatives R_i for the quotient space $G^{(E_{i-1})}/G^{(E_i)}$ by setwise stabilizing the columns in E_{i-1} and searching for automorphisms that map E_i to some E_j for all larger indices $t_j \in B'$ in the orbit X of t_i . By testing only the unique representatives of each redundancy class, the algorithm evaluates exactly one target per class, avoiding redundant oracle queries.

Any automorphism that maps E_i to E_j maps all copies of column $\mathbf{g}_{t_i}$ to all copies of column $\mathbf{g}_{t_j}$, where $t_i, t_j \in B'$. To test for the existence of such automorphisms, we use a similar technique to that in [1, 4] and augment the matrix $\mathbf{G}$ with copies of columns to ensure unique frequency. In particular, we construct test matrices $\mathbf{G}_A$ and $\mathbf{G}_B$ by appending blocks of $(l+1)m$ copies of $\mathbf{g}_{t_i}$ and $(l+1)m$ copies of $\mathbf{g}_{t_j}$, respectively, to the current stabilized matrix in which column classes $E_1, \dots, E_{i-1}$ are fixed setwise. By definition of m and l , no other column in $\mathbf{G}_A$ and $\mathbf{G}_B$ appears the same number of times, so $\mathbf{g}_{t_i}$ and $\mathbf{g}_{t_j}$ have the same unique frequency. By Lemma 2.1, the PCE oracle can only respond YES if there is an automorphism that maps E_i to E_j . Thus, the oracle is forced to decide equivalence exclusively within the stabilizer subgroup $G^{(E_{i-1})}$.

If an automorphism that maps E_i to E_j exists, then the algorithm calls the search-PCE oracle to find it. It outputs a map π that permutes the columns of the current fixed matrix $\mathbf{G}_A$, which has at least $n + (l+1)m \geq n+m$ columns in any given iteration. We derive a permutation $\sigma \in \mathcal{S}_n$ on the original index set by extracting the images $\pi(i)$ for all $i \in [n]$. Any coordinate c that maps into the desired range $[n]$ is preserved; if c is otherwise mapped to a larger index $\pi(c) > n$, the algorithm identifies the redundancy class of the target $\pi(c)$ and finds the coordinate in $[n]$ of the corresponding identical column. To ensure that no collisions are created among the image elements of σ , the algorithm tracks all unassigned image values with the set $U \subseteq [n]$ and assigns $\sigma(c)$ to be the smallest available coordinate $t' \in U$. This ensures that σ is a bijection on the index set $[n]$. Since the algorithm finds an automorphism for every valid pair E_i and E_j , it finds all coset representatives R_i for the quotient space $G^{(E_{i-1})}/G^{(E_i)}$, for all $i \in [s]$. Thus, the final generating set S contains $R_1, \dots, R_s$ and a generating set S_{E_s} for $G^{(E_s)}$. Therefore, by Lemma 3.14, this S is a strong generating set for the entire automorphism group G .

(Complexity) We first bound the number of oracle calls. The algorithm makes a single query to the APART oracle to obtain the orbit partition of the index set $[n]$. In the main for-loop, a target $t_j \in B'$ is tested against a base coordinate $t_i \in B'$ only if both belong to the same orbit X . For any orbit X , the number of distinct column classes is bounded by $|X|$. Thus, the outer for-loop evaluates the test branch at most $|X|$ times for this orbit, and each inner for-loop tests at most $|X| - 1$ targets. This limits the number of PCE oracle queries to $O(|X|^2)$ for each X . Because the coordinate orbits form a partition of $[n]$, the sum of $O(|X|^2)$ across all orbits X is bounded above by $O(n^2)$. Therefore, the algorithm makes a total of $O(n^2)$ calls to the PCE oracle. The Search-PCE oracle is invoked only when the existence of an automorphism is confirmed by the PCE oracle, and so is invoked at most $O(n^2)$ times across the entire algorithm.

Now we bound the runtime. Partitioning the index set $[n]$ into redundancy classes and constructing the set B' requires comparing column vectors at most n^2 times, so this step requires $O(n^2 \log(n))$ time. By Proposition 4.3, running the APART oracle requires $O(n^3 \log(q))$ time. By Lemma 1.1, running the search-PCE oracle requires $O(n^3 \log(q))$ time. Now we evaluate the cost of constructing the test matrices, which dominates the time complexity of the last step and the entire algorithm. The maximum column multiplicity in $\mathbf{G}$ is $m \leq n$, and in each iteration $t_i \in B'$, the number of unique column classes fixed so far is $l \leq n$. The matrix $\mathbf{G}_{\text{fixed}}$ of setwise stabilized columns accumulates $\sum_{a=1}^l a \cdot m$ appended columns in each iteration. To construct $\mathbf{G}_A$ and $\mathbf{G}_B$, an additional block of $(l+1)m$ columns is appended to $\mathbf{G}_{\text{fixed}}$. Thus, the number of columns appended to $\mathbf{G}$ is bounded above by $n + ml(l+1)/2 + (l+1)m = O(n^3)$.

Each field element in $\mathbb{F}_q$ requires $\log(q)$ bits to represent and $O(\log(q))$ bit operations to read, copy, or compare. The initial partitioning phase and computing the size of $G^{(E_s)}$ requires $O(n^2 \log(q))$ and $O(n)$ operations respectively. Then, for each of the $O(n^2)$ oracle queries, copying at most $O(n^3)$ column vectors for the augmented matrices requires $O(kn^3 \log(q))$ operations. If an automorphism π is found by the oracle, constructing the permutation σ requires $O(n)$ local comparisons. Therefore, since $k \leq n$, the total runtime is bounded by $O(n^2) \cdot O(kn^3 \log(q)) = O(n^6 \log(q))$ time. $\square$

6.4 MAGEN Reduces to LCE

Now we prove the monomial analog of [Proposition 6.3](#). The reduction algorithm is nearly identical to [Algorithm 12](#) but uses the properties of monomial equivalence and an LCE oracle instead of PCE. For completeness, we include the full algorithm and proof below.

Proposition 6.4. *Algorithm 13 correctly solves MAGEN for any n -length q -ary code in $O(n^6 \log(q))$ time using one call to an MAPART oracle, $O(n^2)$ queries to a search-LCE oracle, and $O(n^2)$ additional calls to an LCE oracle.*

Proof The proof is structurally identical to that of [Proposition 6.3](#), but using the setwise stabilizer chain framework for the monomial automorphism group. Let $M \geq M^{(E_1)} \geq \dots \geq M^{(E_s)}$ be the monomial setwise stabilizer chain for input generator matrix $\mathbf{G}$, as in [Lemma 3.13](#). The algorithm here finds monomial generators of the smallest subgroup $M^{(E_s)}$ by independently scaling the c indecomposable summands of $\mathcal{C}(\mathbf{G})$ and adjusting the internal transpositions. For each oracle call to search-LCE, the oracle finds a linear isometry for the augmented code that maps a column i to another column j . The algorithm extracts a linear isometry for the original code $\mathcal{C}(\mathbf{G})$ with index set $[n]$ by remapping any out-of-bounds image coordinates as before, but now it multiplies the associated scalar by the corresponding column ratio λ .

The number of oracle calls is identical to that of [Algorithm 12](#): the MAPART oracle is called once; the number of LCE oracle calls is bounded by the $O(n^2)$, the number of candidate test coordinate-target pairs; and the number of search-LCE oracle calls is bounded by $O(n^2)$, the number of positive responses from the PCE oracle. For the time complexity, there are additional scalar operations, which take $O(\log(q))$ time in each loop. The column ratio λ is also computed at most twice for each coordinate in $[n]$; since λ can be obtained by dividing two column entries with the same index, this requires at most $O(n \cdot \log(n))$ time. Hence, the total runtime remains bounded by $O(n^6 \log(q))$. $\square$

Algorithm 13: Reduction from MAGEN to LCE

Input : A generator matrix $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ of a linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$.

Output: A generating set $S \subseteq \mathcal{S}_n$ for the monomial automorphism group $\text{MAut}(\mathcal{C})$.

```
// Step 1: Partition index set into projective redundancy classes.
Let  $\{\mathbf{g}_1, \dots, \mathbf{g}_n\}$  be the set of column vectors of  $\mathbf{G}$ .
 $\mathcal{E} := \emptyset$ . // collection of redundancy sets
 $\mathcal{G} := \emptyset$ . // keep track of unique projective column representatives
 $B' := \emptyset$ . // indices of first appearances
for  $i \leftarrow 1$  to  $n$  do
    // If a column is a scalar multiple of a previously seen column, sort it into
    // the same class. Otherwise, create a new class.
    if  $\exists \mathbf{v} \in \mathcal{G}$  such that  $\mathbf{g}_i = \lambda \mathbf{v}$  for some  $\lambda \in \mathbb{F}_q^*$  then
        for  $E \in \mathcal{E}$  do
             $\mathbf{v} \leftarrow E[1]$ .
            if  $\mathbf{g}_i = \lambda \mathbf{v}$  for some  $\lambda \in \mathbb{F}_q^*$  then
                 $E \leftarrow E \cup \{\mathbf{g}_i\}$ .
        else
             $E := \{\mathbf{g}_i\}$ .
             $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ 
             $\mathcal{E} \leftarrow \mathcal{E} \cup \{E\}$ 
             $B' \leftarrow B' \cup \{i\}$ . // record index of first appearance
     $m \leftarrow 0$ . // maximum projective column multiplicity in  $\mathbf{G}$ 
    for  $E \in \mathcal{E}$  do
        if  $|E| > m$  then
             $m \leftarrow |E|$ 
```

(continued on next page)

(continued from previous page)

// Step 2: Build smallest subgroup $M^{(E_s)}$ from global scaling and internal swaps.
 Run [Algorithm 1](#) on $\mathbf{G}$ to obtain the decomposition $\pi(\mathcal{C}) = \bigoplus_{j=1}^c A_j$, for some indecomposable codes $\{A_j\}_{j=1}^c$ and permutation $\pi \in \mathcal{S}_n$.

$S \leftarrow \emptyset$. // initialize generating set

$\alpha :=$ a primitive element of $\mathbb{F}_q^*$.

for $j \leftarrow 1$ **to** c **do**

 // Generator for independent scaling of summand A_j

$\sigma = (\mathbf{v}_\sigma, \pi_\sigma) := (\mathbf{1}, \text{id})$.

for $u \in \text{supp}(A_j)$ **do**

$\mathbf{v}_\sigma[u] \leftarrow \alpha$.

$S \leftarrow S \cup \{\sigma\}$.

for $E \in \mathcal{E}$ **do**

$\mathbf{g}_{e_1} \leftarrow E[1]$.

for $\mathbf{g}_{e_i} \in E$ **where** $\mathbf{g}_{e_i} \neq \mathbf{g}_{e_1}$ **do**

 // Generator for internal swap uniquely determined by scalars

$\sigma = (\mathbf{v}_\sigma, \pi_\sigma) := (\mathbf{1}, \text{id})$.

$\pi_\sigma(e_1) \leftarrow e_i$

$\pi_\sigma(e_i) \leftarrow e_1$

 Find $\lambda \in \mathbb{F}_q^*$ such that $\mathbf{g}_{e_i} = \lambda \mathbf{g}_{e_1}$.

$\mathbf{v}_\sigma[e_1] \leftarrow \lambda$

$\mathbf{v}_\sigma[e_i] \leftarrow \lambda^{-1}$

$S \leftarrow S \cup \{\sigma\}$.

// Step 3: Find the monomial orbit partition via MAPART oracle.

Run MAPART($\mathbf{G}$) to obtain monomial coordinate orbits $X_1, \dots, X_\ell$.

// Step 4: Compute all coset transversals $\sigma \in R_i$ along the stabilizer chain.

$\mathcal{G} \leftarrow \emptyset$. // track permanently fixed columns in subchain

$\mathbf{G}_{\text{fixed}} \leftarrow \mathbf{G}$. // initialize global base matrix

(continued on next page)

(continued from previous page)

```

for  $i \in B'$  do
   $X :=$  unique orbit in  $\{X_1, \dots, X_\ell\}$  containing  $i$ .
  // Find a mapping from  $E_i$  to each valid target class.
  for  $j \in X \cap B'$  where  $j > i$  do
     $l \leftarrow |\mathcal{G}|$ .
     $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
     $\mathbf{B} := (l+1)m$  copies of  $\mathbf{g}_j$ .
     $\mathbf{G}_A \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ .
     $\mathbf{G}_B \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{B}]$ .

    // Check if there exists an automorphism that maps  $i$  to  $j$ .
    if  $\text{LCE}(\mathbf{G}_A, \mathbf{G}_B) \rightarrow \text{YES}$  then
      // If an automorphism exists, find it via the Search-LCE oracle.
      Run  $\text{Search-LCE}(\mathbf{G}_A, \mathbf{G}_B)$  to obtain a linear isometry  $\mu = (\mathbf{v}_\mu, \pi_\mu)$ .
       $U \leftarrow \{1, \dots, n\}$ . // keep track of mapped indices
       $\sigma = (\mathbf{v}_\sigma, \pi_\sigma) := (\mathbf{1}, \text{id})$ . // initialize generator

      for  $c \leftarrow 1$  to  $n$  do
        // Identify the original column  $t$  that the oracle mapped to.
        if  $\pi_\mu(c) \leq n$  then
           $t := \pi_\mu(c)$ 
        else
           $t :=$  original coordinate in  $[n]$  that the padded column  $\pi_\mu(c)$  is a copy of.
        // Find an available projectively equivalent target  $t'$  in  $U$ .
         $t' := \min\{u \in U \mid \mathbf{g}_u = \lambda \mathbf{g}_t \text{ for some } \lambda \in \mathbb{F}_q^*\}$ 
        Find  $\lambda \in \mathbb{F}_q^*$  such that  $\mathbf{g}_{t'} = \lambda \mathbf{g}_t$ .
        // Redirect the map and adjust the scalar.
         $\pi_\sigma(c) \leftarrow t'$ 
         $\mathbf{v}_\sigma[c] \leftarrow \lambda \cdot \mathbf{v}_\mu[c]$ 
         $U \leftarrow U \setminus \{t'\}$ 
       $S \leftarrow S \cup \{\sigma\}$ . // add transversal representative

   $\mathbf{A} := (l+1)m$  copies of  $\mathbf{g}_i$ .
   $\mathbf{G}_{\text{fixed}} \leftarrow [\mathbf{G}_{\text{fixed}} \mid \mathbf{A}]$ . // fix base point for next iteration
   $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathbf{g}_i\}$ .

return  $S$ .
```

7 Conclusion

We show that CE is a polynomially equivalent problem to several computational problems relating to the automorphism group of a code. These problems ask for the cardinality, an orbit partition, and a generating set of the automorphism group of a given code, and are called **ACOUNT**, **APART**, and **AGEN**, respectively. The relationship between these problems provides a way to solve the permutation (PCE) and monomial (LCE) variants of CE, given some information about the automorphism groups of the input codes, and vice versa. A key component of our reductions is a method of decomposing any linear code into indecomposable codes; this framework provides a way of studying permutation and monomial equivalence, and may be of independent interest to other areas of coding theory. For future work, it remains to see how our reductions can inform the construction and analysis of cryptosystems built on Code Equivalence. Given the similarity between CE and LIP, it would also be interesting to study analogous problems for lattices.

References

- [1] Biasse, J.-F., Micheli, G.: A Search-to-Decision Reduction for the Permutation Code Equivalence Problem. In: 2023 IEEE International Symposium on Information Theory (ISIT), pp. 602–607 (2023). <https://doi.org/10.1109/ISIT54713.2023.10206940>
- [2] Biasse, J.-F., Micheli, G., Prada, B., Waitkevich, P.: A search-to-decision reduction for the linear code equivalence problem (2026). <https://arxiv.org/abs/2609.31517>
- [3] Mazumder, A.: Search-to-Decision Reductions for the Linear and General Code Equivalence Problems (2026). <https://arxiv.org/abs/2608.10967>
- [4] Cheraghchi, M., Shagrirhaya, N., Veliche, A.: Reductions Between Code Equivalence Problems. In: 2025 IEEE International Symposium on Information Theory (ISIT), pp. 1–6 (2025). <https://doi.org/10.1109/ISIT63088.2025.11195558>
- [5] Horlemann, A.-L., Mazumder, A., Schaller, M., Weger, V.: A survey on code equivalence: The state-of-the-art and open questions. In: International Workshop on the Arithmetic of Finite Fields, pp. 12–31 (2026). Springer
- [6] Petrank, E., Roth, R.M.: Is Code Equivalence Easy to Decide? IEEE Trans. Inf. Theor. **43**(5), 1602–1604 (2006) <https://doi.org/10.1109/18.623157>
- [7] Kaski, P., Östergård, P.R.J.: Representations and Isomorphism, pp. 47–104. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/3-540-28991-7_3 . https://doi.org/10.1007/3-540-28991-7_3
- [8] Grochow, J.A.: Matrix isomorphism of matrix lie algebras. In: Proceedings of the 2012 IEEE Conference on Computational Complexity (CCC). CCC '12, pp. 203–213. IEEE Computer Society, USA (2012). <https://doi.org/10.1109/CCC.2012.34> . <https://doi.org/10.1109/CCC.2012.34>
- [9] Bennett, H., Win, K.M.H.: Relating Code Equivalence to Other Isomorphism Problems. Cryptology ePrint Archive, Paper 2024/782 (2024). <https://eprint.iacr.org/2024/782>
- [10] Mathon, R.: A Note on the Graph Isomorphism Counting Problem. Information Processing Letters **8**(3), 131–136 (1979) [https://doi.org/10.1016/0020-0190\(79\)90004-8](https://doi.org/10.1016/0020-0190(79)90004-8)
- [11] Girault, M.: A (non-practical) three-pass identification protocol using coding theory. In: Seberry, J., Pieprzyk, J. (eds.) Advances in Cryptology — AUSCRYPT '90, pp. 265–272. Springer, Berlin, Heidelberg (1990)
- [12] Sendrier, N., Simos, D.E.: The hardness of code equivalence over and its application to code-based cryptography. In: International Workshop on Post-Quantum Cryptography, pp. 203–216 (2013). Springer
- [13] Barenghi, A., Biasse, J., Ngo, T., Persichetti, E., Santini, P.: Advanced signature functionalities from the code equivalence problem. Int. J. Comput. Math. Comput. Syst. Theory **7**(2), 112–128 (2022) <https://doi.org/10.1080/23799927.2022.2048206>
- [14] Barenghi, A., Biasse, J., Persichetti, E., Santini, P.: LESS-FM: fine-tuning signatures from the code equivalence problem. In: Cheon, J.H., Tillich, J. (eds.) Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings. Lecture Notes in Computer Science, vol. 12841, pp. 23–43. Springer, ??? (2021). https://doi.org/10.1007/978-3-030-81293-5_2 . https://doi.org/10.1007/978-3-030-81293-5_2
- [15] Biasse, J.-F., Micheli, G., Persichetti, E., Santini, P.: Less is more: Code-based signatures without syndromes. In: Progress in Cryptology - AFRICACRYPT 2020: 12th International Conference on Cryptology in Africa, Cairo, Egypt, July 20 – 22, 2020, Proceedings, pp. 45–65. Springer,

- Berlin, Heidelberg (2020). https://doi.org/10.1007/978-3-030-51938-4_3 . https://doi.org/10.1007/978-3-030-51938-4_3
- [16] Ducas, L., Woerden, W.P.J.: On the lattice isomorphism problem, quadratic forms, remarkable lattices, and cryptography. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 13277, pp. 643–673. Springer, ??? (2022). https://doi.org/10.1007/978-3-031-07082-2_23 . https://doi.org/10.1007/978-3-031-07082-2_23
 - [17] Ducas, L., Postlethwaite, E.W., Pulles, L.N., Woerden, W.P.J.: Hawk: Module LIP makes lattice signatures fast, compact and simple. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security*, Taipei, Taiwan, December 5–9, 2022, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 13794, pp. 65–94. Springer, ??? (2022). https://doi.org/10.1007/978-3-031-22972-5_3 . https://doi.org/10.1007/978-3-031-22972-5_3
 - [18] Barengi, A., Biasse, J., Persichetti, E., Santini, P.: On the computational hardness of the code equivalence problem in cryptography. *Adv. Math. Commun.* **17**(1), 23–55 (2023) <https://doi.org/10.3934/AMC.2022064>
 - [19] D’Alconzo, G.: Monomial isomorphism for tensors and applications to code equivalence problems. *Des. Codes Cryptogr.* **92**(7), 1961–1982 (2024) <https://doi.org/10.1007/S10623-024-01375-0>
 - [20] Budroni, A., Chi-Domínguez, J., D’Alconzo, G., Scala, A.J.D., Kulkarni, M.: Don’t use it twice! solving relaxed linear equivalence problems. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9–13, 2024, Proceedings, Part VIII. *Lecture Notes in Computer Science*, vol. 15491, pp. 35–65. Springer, ??? (2024). https://doi.org/10.1007/978-981-96-0944-4_2 . https://doi.org/10.1007/978-981-96-0944-4_2
 - [21] Chou, T., Persichetti, E., Santini, P.: On linear equivalence, canonical forms, and digital signatures. *Des. Codes Cryptogr.* **93**(7), 2415–2457 (2025) <https://doi.org/10.1007/S10623-025-01576-1>
 - [22] Bennett, H., Bhatia, D., Biasse, J., Durisheti, M., LaBuff, L., Lavorante, V.P., Waitkevich, P.: Asymptotic improvements to provable algorithms for the code equivalence problem. In: *IEEE International Symposium on Information Theory, ISIT 2025, Ann Arbor, MI, USA, June 22–27, 2025*, pp. 1–6. IEEE, ??? (2025). <https://doi.org/10.1109/ISIT63088.2025.11195271> . <https://doi.org/10.1109/ISIT63088.2025.11195271>
 - [23] Nowakowski, J.: An improved algorithm for code equivalence. In: Niederhagen, R., Saarinen, M.O. (eds.) *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Taipei, Taiwan, April 8–10, 2025, Proceedings, Part I*. *Lecture Notes in Computer Science*, vol. 15577, pp. 71–103. Springer, ??? (2025). https://doi.org/10.1007/978-3-031-86599-2_3 . https://doi.org/10.1007/978-3-031-86599-2_3
 - [24] Bennett, H., Bhatia, D., Biasse, J., Durisheti, M., LaBuff, L., Lavorante, V.P., Waitkevich, P.: Asymptotic improvements to provable algorithms for the code equivalence problem. *IEEE Trans. Inf. Theory* **72**(2), 1093–1108 (2026) <https://doi.org/10.1109/TIT.2025.3637683>
 - [25] Bardet, M., Brion, C., Otmani, A., Saeed, M., Sendrier, N.: Solving the linear equivalence problem from single codeword matching. In: Heninger, N., Rosulek, M. (eds.) *Advances in Cryptology - CRYPTO 2026 - 46th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 17–20, 2026, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 16803, pp. 256–288. Springer, ??? (2026). https://doi.org/10.1007/978-3-032-35398-6_9 . https://doi.org/10.1007/978-3-032-35398-6_9

- [26] Battagliola, M., Mora, R., Santini, P.: Using the schur product to solve the code equivalence problem. *IEEE Trans. Inf. Theory* **72**(7), 4675–4694 (2026) <https://doi.org/10.1109/TIT.2026.3694630>
- [27] Battagliola, M., Horlemann, A., Mazumder, A., Mora, R., Santini, P., Schaller, M., Weger, V.: The power of power codes: New classes of easy instances for the linear equivalence problem. In: *IEEE International Symposium on Information Theory, ISIT 2026, Guangzhou, China, June 28 - July 3, 2026*, pp. 1–6. IEEE, ??? (2026). <https://doi.org/10.1109/ISIT62367.2026.11653777> . <https://doi.org/10.1109/ISIT62367.2026.11653777>
- [28] Budroni, A., Esser, A.: One pair to rule them all: Towards an optimal algorithm for solving code equivalence via codeword search. In: Heninger, N., Rosulek, M. (eds.) *Advances in Cryptology - CRYPTO 2026 - 46th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-20, 2026, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 16803, pp. 319–346. Springer, ??? (2026). https://doi.org/10.1007/978-3-032-35398-6_11 . https://doi.org/10.1007/978-3-032-35398-6_11
- [29] Budroni, A., Esser, A., Franch, E., Natale, A.: Two is all it takes: Asymptotic and concrete improvements for solving code equivalence. In: Bai, S., Persichetti, E. (eds.) *Public-Key Cryptography - PKC 2026 - 29th IACR International Conference on Practice and Theory of Public-Key Cryptography, West Palm Beach, FL, USA, May 25-28, 2026, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 16551, pp. 395–427. Springer, ??? (2026). https://doi.org/10.1007/978-3-032-26731-3_13 . https://doi.org/10.1007/978-3-032-26731-3_13
- [30] Alecci, G., D’Alconzo, G.: Linear code equivalence via plücker coordinates. *CoRR* **abs/2603.09869** (2026) <https://doi.org/10.48550/ARXIV.2603.09869> 2603.09869
- [31] Baldi, M., Barengi, A., Beckwith, L., Biasse, J.-F., Esser, A., Gaj, K., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.-J.O., Santini, P., Wallace, R.: LESS. Additional Digital Signature Schemes - Round 2 Submissions (2023). <https://www.less-project.com/>
- [32] Bos, J.W., Bronchain, O., Ducas, L., Fehr, S., Huang, Y.-H., Pornin, T., Postlethwaite, E.W., Prest, T., Pulles, L.N., Woerden, W.: HAWK. Additional Digital Signature Schemes - Round 1 Submissions (2023). <https://hawk-sign.info/>
- [33] Grochow, J.: Complexity of finding automorphism group of code. *Theoretical Computer Science Stack Exchange* (February 5, 2020). <https://cstheory.stackexchange.com/questions/46296/complexity-of-finding-automorphism-group-of-code>
- [34] Bogart, K., Goldberg, D., Gordon, J.: An Elementary Proof of the MacWilliams Theorem on Equivalence of Codes. *Information and Control* **37**(1), 19–22 (1978) [https://doi.org/10.1016/S0019-9958\(78\)90389-3](https://doi.org/10.1016/S0019-9958(78)90389-3)
- [35] Sims, C.C.: Computational Methods in the Study of Permutation Groups. In: *Computational Problems in Abstract Algebra*, pp. 169–183 (1970)
- [36] Seress, Á.: *Permutation Group Algorithms*. Cambridge Tracts in Mathematics. Cambridge University Press, ??? (2003)
- [37] Slepian, D.: Some Further Theory of Group Codes. *Bell System Technical Journal* **39**(5), 1219–1252 (1960) <https://doi.org/10.1002/j.1538-7305.1960.tb03958.x> <https://onlinelibrary.wiley.com/doi/pdf/10.1002/j.1538-7305.1960.tb03958.x>
- [38] Sendrier, N., Simos, D.E.: How easy is code equivalence over $\mathbb{F}_q$? In: *International Workshop on Coding and Cryptography - WCC 2013, Bergen, Norway* (2013). <https://inria.hal.science/hal-00790861>
- [39] Kung, J.P.S.: Matroid decomposition (k. truemper). *SIAM Review* **35**(3), 529–531 (1993) <https://doi.org/10.1137/350529>

[//doi.org/10.1137/1035123](https://doi.org/10.1137/1035123) <https://doi.org/10.1137/1035123>

- [40] Kashyap, N.: A decomposition theory for binary linear codes. IEEE Transactions on Information Theory **54**(7), 3035–3058 (2008) <https://doi.org/10.1109/TIT.2008.924706>
- [41] Hoffmann, C.M.: Group-theoretic Algorithms and Graph Isomorphism. Springer, ??? (1982). https://link.springer.com/chapter/10.1007/3-540-11493-9_2
- [42] Hopcroft, J.E., Karp, R.M.: An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs. SIAM Journal on Computing **2**(4), 225–231 (1973) <https://doi.org/10.1137/0202019> <https://doi.org/10.1137/0202019>